Vault Field Manual

Sachin

© 2026 v1 Sachin Kansal . All rights reserved.

Table of Contents

1. Introduction 3
   - Why Vault 3
   - Architecture - Internals 5
2. Installation & Configuration 8
   - Vault Production Setup 8
   - Vault Auto Unseal GCP 11
3. Tokens 15
   - Tokens Overview 15
   - Components of Tokens 15
   - Token Life Cycle 16
   - Service Tokens vs Batch Tokens 20
   - Root token 22
4. Auth Methods 25
   - Kubernetes authentication agent injector 25
   - UserPass authentication 29
5. Secrets Engines 33
   - Key/Value pair 33
   - Hashicorp VAULT SSH ENGINE 44
   - Dynamic Database Secrets 47
6. Operations 51
   - Migrating Storage Backend 51

© 2026 v1 Sachin Kansal . All rights reserved.

1. Introduction

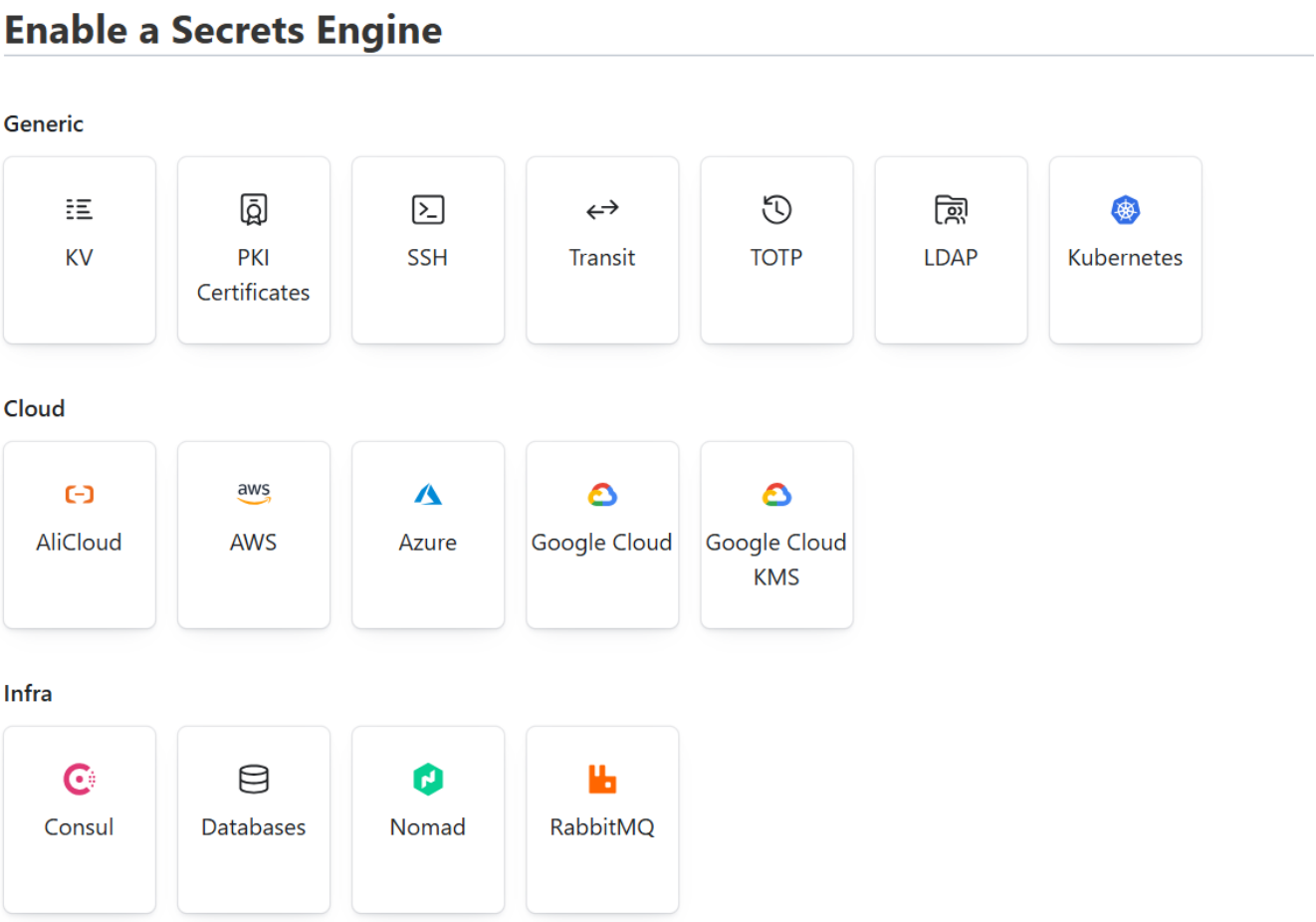
Why Vault

Many organisations use Vault exclusively for the KV secrets engine — storing certificates, environment files, and static database credentials. When asked why they chose Vault, most engineers give the same answers: security, reusability of secrets, and prevention of secret sprawl. These are genuine and important reasons.

But using Vault only for KV is like using an AI agent to do third grade maths. You are paying the full cost of complexity and getting a fraction of the value.

Vault provides authentication integrations with almost every major platform — AWS, GCP, Azure, Kubernetes, Okta, GitHub, and more. Beyond KV, its secrets engines include:

- **PKI** — issue and rotate TLS certificates automatically
- **SSH** — short-lived signed certificates instead of static keys
- **Database** — dynamic credentials that expire after use
- **Kubernetes** — pod-level identity and secret injection
- **And More**



We can also create our own custom Secret Engine if the requrements are not met by the provided ones.

Add to this just-in-time cloud access, audit logs, Prometheus telemetry, and automatic failover — and the picture changes completely.

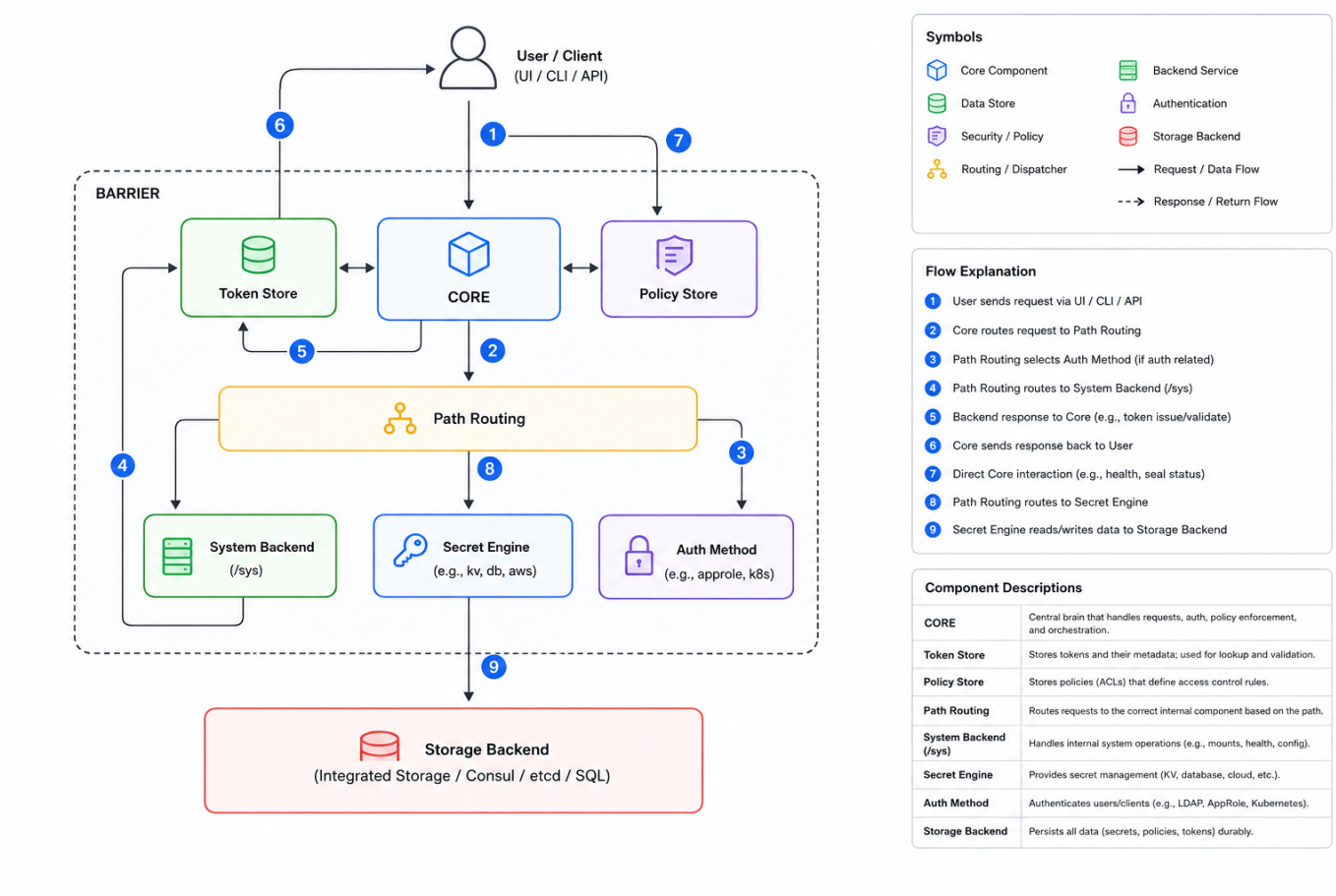
© 2026 v1 Sachin Kansal . All rights reserved.

Used together, these features transform Vault from a password manager into a complete security platform — one that handles identity, access, encryption, and audit across your entire infrastructure.

We are gonna explore the each of these system and what are thier caveats

© 2026 v1 Sachin Kansal . All rights reserved.

Architecture - Internals



Component Descriptions

CORE	Central brain that handles requests, auth, policy enforcement, and orchestration.
Token Store	Stores tokens and their metadata; used for lookup and validation.
Policy Store	Stores policies (ACLs) that define access control rules.
Path Routing	Routes requests to the correct internal component based on the path.
System Backend (/sys)	Handles internal system operations (e.g., mounts, health, config).
Secret Engine	Provides secret management (KV, database, cloud, etc.).
Auth Method	Authenticates users/clients (e.g., LDAP, AppRole, Kubernetes).
Storage Backend	Persists all data (secrets, policies, tokens) durably.

Important Components

CORE

The central orchestrator of Vault. Every request passes through Core — it handles token validation, policy enforcement, and coordinates between all other components. It is the only component that communicates directly with the Storage Backend, and it sits behind the encryption barrier, meaning nothing reaches it until Vault is unsealed.

Token Store

Stores all active tokens and their metadata (TTL, policies, parent token). Every successful login produces a token, and that token is validated by Core on every subsequent request. Tokens can have a parent-child hierarchy — revoking a parent cascades to all its children.

Policy Store

Stores all HCL policy objects. Policies define what capabilities (read, write, delete, list, sudo) are allowed on which paths. They follow a default-deny model — if a path isn't explicitly allowed, access is denied. Policies are attached to tokens at login time and checked by Core on every request.

Path Routing

Routes incoming requests to the correct backend based on the request path. Vault maintains a mount table that maps path prefixes to backends — for example, `secret/` to a KV engine, `auth/kubernetes/` to the Kubernetes auth method. Path Routing consults this table and dispatches accordingly.

© 2026 v1 Sachin Kansal . All rights reserved.

Auth Methods

Pluggable authentication backends. The right method depends on who or what is authenticating:

Client type	Typical auth method
Human users	Userpass, Okta, LDAP
Kubernetes workloads	Kubernetes
Applications / CI	AppRole
Cloud workloads	AWS, GCP, Azure

On success, the auth method returns an identity back to Core, which then mints a token with the appropriate policies attached.

Secret Engines

Backends that handle secret lifecycle. Each engine is mounted at a path and operates independently. Common types:

- **KV** — static key-value storage (v1 and v2)
- **Database** — dynamic credentials with short TTLs
- **PKI** — certificate issuance
- **SSH** — signed SSH certificates or OTPs
- **Kubernetes** — dynamic service account tokens

Dynamic engines generate credentials on demand and revoke them automatically on lease expiry — no long-lived credentials sitting in config files.

System Backend

A built-in backend mounted at `/sys`. This is where Vault's own configuration lives — mount management, audit device config, seal/unseal operations, health endpoints, and license info. Operators interact with `/sys` to manage the Vault cluster itself, not secrets.

Storage Backend

The physical persistence layer. Vault encrypts everything before writing here — the storage backend never sees plaintext. The backend choice affects availability and performance, not security.

Supported options:

- **Integrated Raft** — built-in, no external dependency, recommended default
- **Consul** — older default, still widely used
- **etcd / SQL** — supported but less common

Switching backends requires `vault operator migrate`.

Request Flow

Two distinct flows depending on what the client is doing:

Getting a token (login):

1. Client sends credentials to an auth path (e.g. `POST /v1/auth/kubernetes/login`)
2. Core forwards to **Path Routing → Auth Method**
3. Auth Method validates credentials against the external provider
4. On success, Core mints a token with policies attached and returns it to the client

© 2026 v1 Sachin Kansal . All rights reserved.

Reading a secret (after login):

1. Client sends request with token (e.g. `GET /v1/secret/data/myapp`)
2. Core validates the token against **Token Store**
3. Core checks the token's policies against **Policy Store** — does this token allow `read` on this path?
4. If allowed, Core routes via **Path Routing** → **Secret Engine**
5. Secret Engine reads from **Storage Backend** (decrypted by Core) and returns data to the client

---

Encryption Barrier

Everything in the diagram inside the dashed boundary sits behind an encryption barrier. The barrier key is loaded at startup via Shamir key shares or auto-unseal (AWS KMS, GCP KMS, etc.). Until unsealed, Vault rejects all requests — a freshly restarted node must be unsealed before it serves traffic.

© 2026 v1 Sachin Kansal . All rights reserved.

2. Installation & Configuration

Vault Production Setup

What We Are Doing ?

We are going to setup a Single-Node Vault server on a Rasberry Pie machine with production like configuration.

For now we are using filesystem as storage backend which is not recommended in production at all. But we will migrate this storage backened later from filesystem to Mysql Database.

When Configuring following points should be kept in mind

- do i need ui
- Run vault server on the Private network with no internet.
- Can Applications and Users/teams can reach it.
- Tls must be enabled for secure communication
- Level of Logs needs to be enabled
- Which Montioring tool are to be used and where to right the logs files

Configure vault.hcl

Full configuration reference → vault.hcl

download and install packages

source :

```
https://developer.hashicorp.com/vault/install
```

```
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?
<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install vault
```

create the Vault User

For least privilidges on linux machine we need to create the vault user. No-one should have any extra access then they need it.

```
 useradd --system --home /etc/vault.d --shell /bin/false vault
```

porper directories

Backend Storage

> we need Data for using filesystem backend storage for now. we will migrate it to DB

```
/srv/vault/data
```

For logs

> We need log directory for keeping the hashicorp vault logs. monitoring tools like filebeat can scrape it.

```
/srv/vault/log
```


© 2026 v1 Sachin Kansal . All rights reserved.

For Audit Logs

> Audit logs to see who/which app logged in when and what they did. No secrets created or changed are captured by the audit logs only actions performed

```
/srv/vault/audit
```

Move binaries from default bin path

```
sudo mv /usr/bin/vault /srv/vault/bin/vault
sudo chown vault:vault /srv/vault/bin/vault
sudo chmod 750 /srv/vault/bin/vault
ln -s /srv/vault/bin/vault /usr/bin/vault
```

configure vault.hcl

- server ip exposing vault for ui 192.168.29.10
- tls

  a. disabled for dev

  ```
  listener "tcp" {
  address = "192.168.29.10:8200"
  tls_disable = 1
  }
  ```

  b. enable for production (must)

  to generate the self signed certificate

  ```
  openssl req -x509 -newkey rsa:4096 \
  -keyout /etc/vault.d/vault-key.pem \
  -out /etc/vault.d/vault-cert.pem \
  -days 365 -nodes \
  -subj "/CN=192.168.29.10" \
  -addext "subjectAltName=IP:192.168.29.10"
  ```

  Cofiguration block

  ```
  #HTTPS listener
  listener "tcp" {
  address       = "192.168.29.10:8200"
  tls_cert_file = "/etc/vault.d/vault-cert.pem"
  tls_key_file  = "/etc/vault.d/vault-key.pem"
  }
  ```

  *** REMEMBER ***

  certificate use must rely on SAN not CN else following error will be thrown by vault cli vault status

  > Error checking seal status: Get "https://192.168.29.10:8200/v1/sys/seal-status": tls: failed to verify certificate: x509: certificate relies on legacy Common Name field, use SANs instead

- telemetry (pre-requiste prometheus)
- create policy for metrics

  ```
  path "sys/metrics" {
  capabilities = ["read","list"]
  }
  ```

- create token with policy attached

  ```
  vault token create \
  -policy=prometheus-metrics \
  -period=720h \
  -display-name=prometheus \
  -no-default-policy
  ```

- update vault and prometheus configs

© 2026 v1 Sachin Kansal . All rights reserved.

vault telemetry

```
#telemetry
telemetry {
prometheus_retention_time = "30s"
disable_hostname = true
}
```

prometheus job

```
- job_name: vault
metrics_path: "/v1/sys/metrics"
scheme: https
tls_config:
  ca_file: "/etc/vault.d/vault-cert.pem"
bearer_token: "hvs.xxxxxxxxxxxxxxxxx"
params:
  format: ['prometheus']
static_configs:

  - targets: ['192.168.29.10:8200']
```

- backened filesystem

```
  # data
  /srv/vault/data
  #logs and audit logs
  /srv/vault/log/vault.log
  chown -R vault:vault /srv/vault/data /srv/vault/log/vault.log
  /srv/vault/audit
```

- mlock enabled

> to prevent keys and tokens from bieng written on the disk

• logging level : Error

systemd unit file

```
cat << EOF > /etc/systemd/system/vault.service
[Unit]
Description=HashiCorp Vault
Documentation=https://www.vaultproject.io/docs
Requires=network-online.target
After=network-online.target

[Service]
User=vault
Group=vault
ExecStart=/srv/vault/bin/vault server -config=/etc/vault.d/vault.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
CapabilityBoundingSet=CAP_IPC_LOCK
AmbientCapabilities=CAP_IPC_LOCK
NoNewPrivileges=yes

[Install]
WantedBy=multi-user.target
EOF
```

init the vault

```
vault status

vault operator init

vault operator unseal  #repeat until sealed false i.e. minimum threshold set

# enable audit logs
chown -R vault:vault /srv/vault/audit/

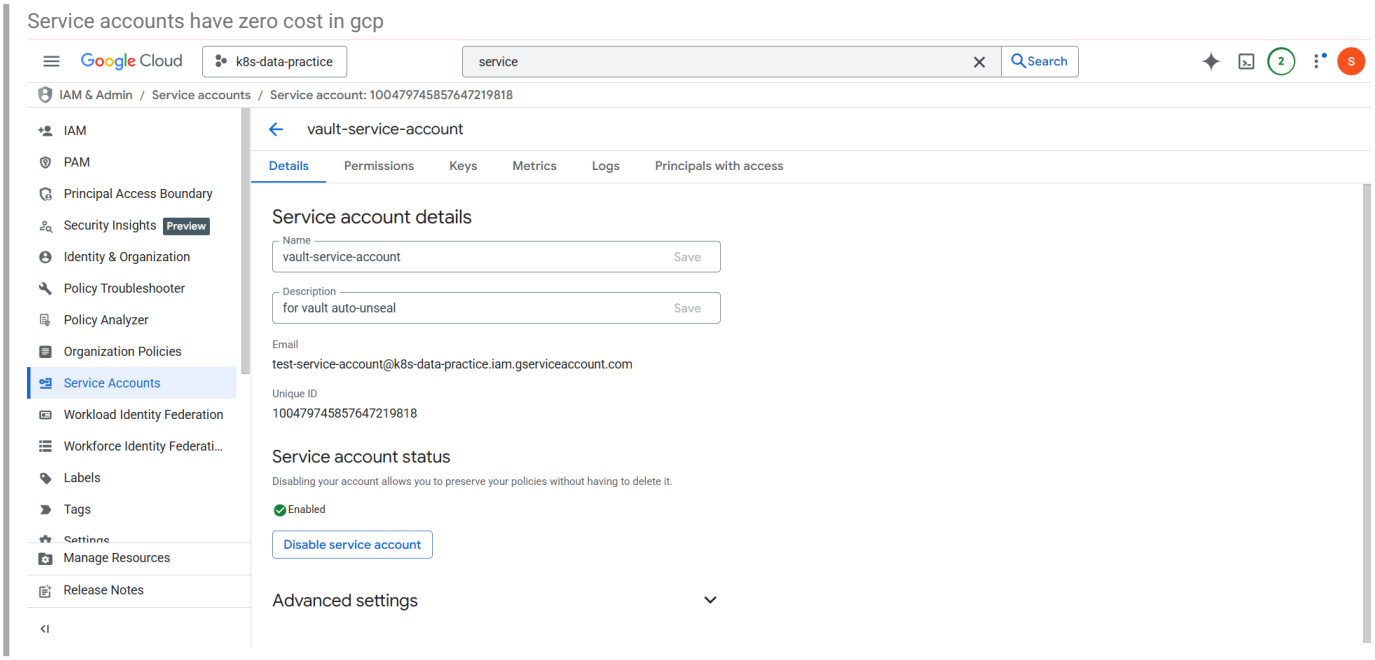
vault audit enable file file_path=/srv/vault/audit/audit.log
```


© 2026 v1 Sachin Kansal . All rights reserved.

Vault Auto Unseal GCP

Migrate shamir sealed vault to gcp autounseal

create a gcp service account



Store the generated service account key on secure location onserver as our vault is on-prem it require to do authentication to cloud using this key.json.

we need to make sure key.json has minimal permission and the ownership to vault

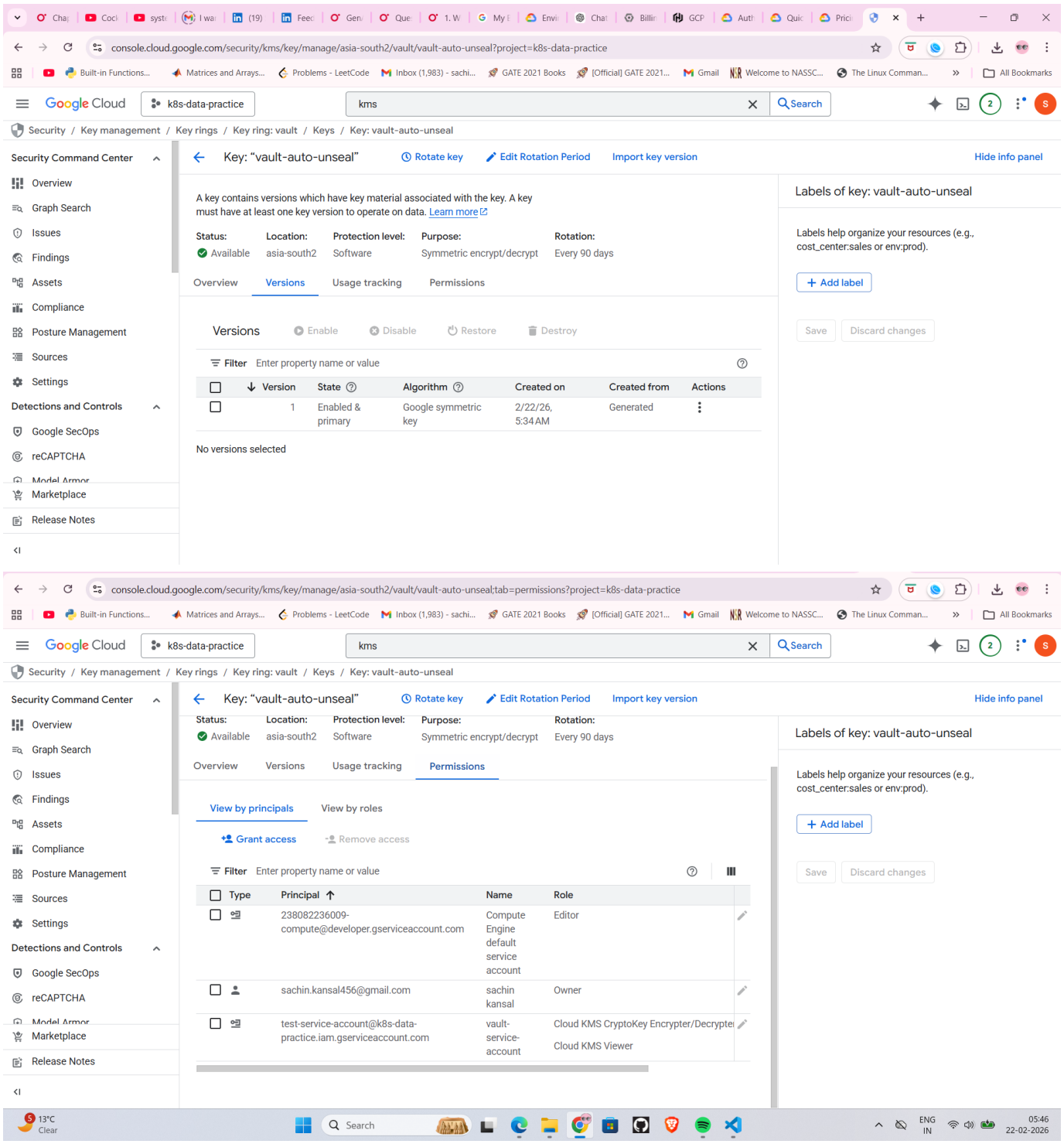
ROLES

We need to grant the following Roles to the created service account to encrypt/decrypt the keys.

- Cloud KMS CryptoKey Encrypter/Decrypter
- Cloud KMS Viewer

© 2026 v1 Sachin Kansal . All rights reserved.

Create key ring and key



update vault.hcl

In Vault.hcl add the following block containing the information regarding how to connect and authenticate to the gcp cloud account and which kms key ring it need to use to generate the seal keys.

```
seal "gcpckms" {
  project     = "k8s-data-practice"
  region      = "asia-south2"
  key_ring    = "vault"
  crypto_key  = "vault-auto-unseal"
```


© 2026 v1 Sachin Kansal . All rights reserved.

```
  credentials = "/srv/vault/key.json"
}
```

restart the vault

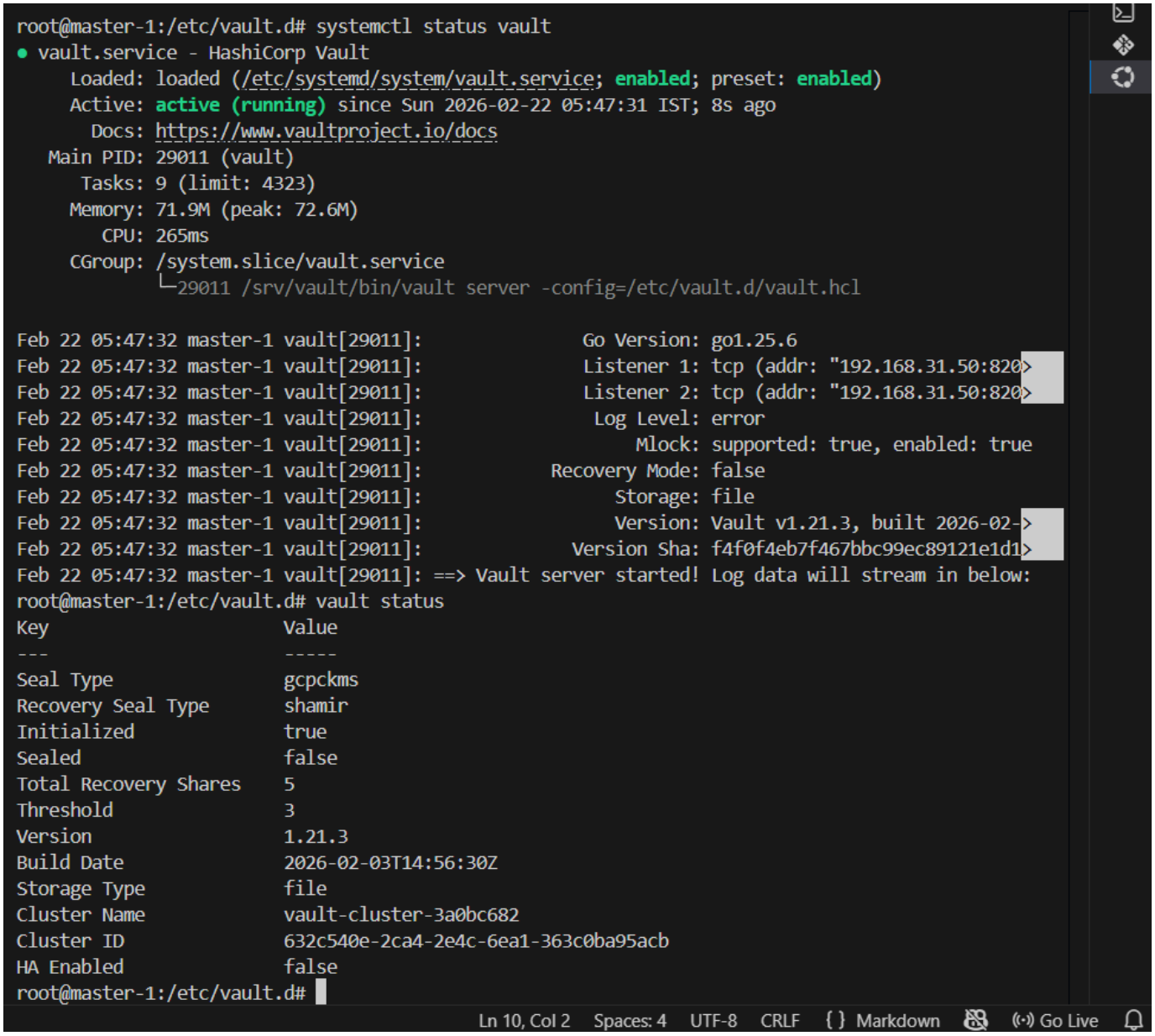
```
systemctl restart vault
```

migrate from shamir to gcpckms

```
vault operator unseal -migrate
```

provide seal keys until thresold is reached and vault is unsealed. this unsealing is one time activity and is required as our storage backened still think of it as shamir.

post migrate restart the cluster and this time it will be auto unsealed



NOTE: you see the recovery keys are still mentioned as they are still valid and useful incase of disater recovery or critical issue.

For each restart the keys are taken from cloud hence the cost of using kms is incurred. to prevent the cost remigrate from auto-unseal to shamir

© 2026 v1 Sachin Kansal . All rights reserved.

downgrade from gcp auto unseal to manual shamir

update config

```
seal "gcpckms" {
  project     = "k8s-data-practice"
  region      = "asia-south2"
  key_ring    = "vault"
  crypto_key  = "vault-auto-unseal"
  credentials = "/srv/vault/key.json"
  disabled = "true"
}

shamir {}
```

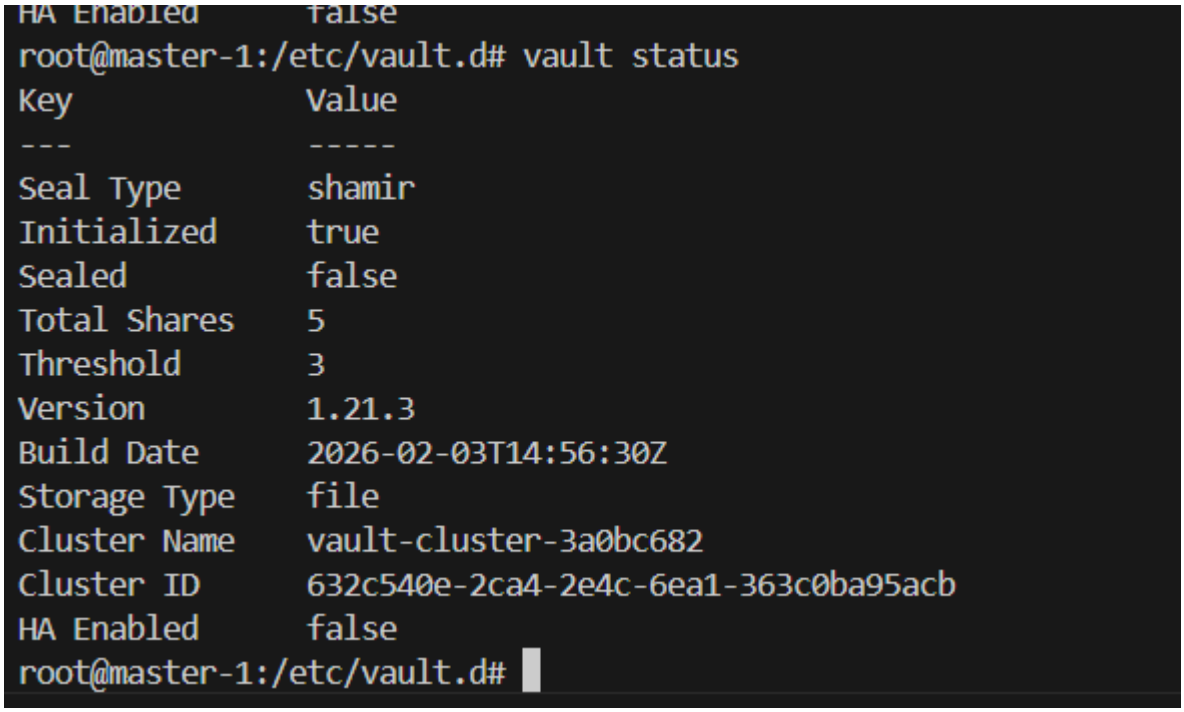
restart the vault using `systemctl restart vault`

`vault operator unseal -migrate` provide keys until threshold i.e. 3

again restart the vault now auto unseal is disable.

remove the gcpckms block from .hcl , delete the key.json file and delete the keyring and key if not required.

As deletion of keys take 30 days of time. you can remove the permission for the keys so no one can use it.



© 2026 v1 Sachin Kansal . All rights reserved.

3. Tokens

Tokens Overview

No token should be left active

Tokens are the primary credentials used by Vault for authorization and session management. While users may authenticate through mechanisms such as UserPass, AppRole, Kubernetes, LDAP, or OIDC, Vault ultimately issues a token that represents the authenticated identity. These tokens are managed by Vault's internal token store, which is responsible for token creation, validation, renewal, and revocation.

```
root@master-1:/home/master# vault login -method=userpass username=sachin
Password (will be hidden):
WARNING! The VAULT_TOKEN environment variable is set! The value of this
variable will take precedence; if this is unwanted please unset VAULT_TOKEN or
update its value accordingly.

Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                  Value
---                  -----
token                hvs.CAESIArLGzlpLsYLH9EBxzwy-Hs3uLTHH0o3gMcHtGU9TImpGh4KHGh2cy5UMDlBMWlpSmViOTNjU1pMbENzRnZsNmw
token_accessor       MhtSZB8xa6aEIJdmalCPC8jb
token_duration       768h
token_renewable      true
token_policies       ["default" "kv-engine"]
identity_policies    []
policies             ["default" "kv-engine"]
token_meta_username  sachin
root@master-1:/home/master#
```

As it can be seen in the image even though user logged in using UserPass mechanism a temporary token is generated which is valid for the user current session, if user re-logins via UI or Api request a new temporary token will be generated. ***This proves the importance of tokens and tokenstore in the vault***

```
root@master-1:/home/master# vault auth disable token/
Error disabling auth method at token/: Error making API request.

URL: DELETE https://192.168.29.10:8200/v1/sys/auth/token
Code: 400. Errors:

* token credential backend cannot be disabled
root@master-1:/home/master#
```

> Token auth method can not be disabled

Components of Tokens

Every token consist of a token_accessor to access the token, the token duration aka time to live the time after which it expires and becomes invalid, policies which define what user/app can do using this token, identity policies if mapped to a identity mentioning the access granted to the identity and other metadata and properties which grant it extra functionality that could be ability to be renewed, dependency on parent token or being a orphan with full independence, how many times it can be used

© 2026 v1 Sachin Kansal . All rights reserved.

property/options defining them are as follows:

- ttl
- type
- use-limit
- role
- renewable
- policy
- period (periodic token) never expire explicit-max-ttl
- orphan
- no-default-policy
- entity-alias
- display-name

The tokens in general can be of various types depending on the UseCase:

1. Service Token
   > used by mostly services and applications
2. Batch Token
   > for applications with high throughput and iops which could impact the overall performance of vault.
3. Root token
   > default admin token generated during init. must be revoked in production setup and regenerate in case of emergencies
4. Periodic Token
   > Tokens which should or needs to be renewed before their expiry
5. Child Token
   > The token derived from some other token(parent token). these tokens gets their default properties from parents which can be override while creation. These token also becomes invalid if parents is revoked so in case of breach rather revoking the multiple child tokens a parent can be revoked and all their children will be revoked in a go.
6. Orphan Token
   > Token who has no parent. these token are independent in nature and does not depend on any other tokens max ttl
7. Wrapped Token
   > Tokens which are wrapped in an encryption. this make sharing the secrets easier and safer as the shared secrets are wrapped in an encryption when reached the end user can be decrypted by them for further use.

Token Life Cycle

1. vault token generated with some policy

© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/mnt/data/vault/files# vault token create -renewable=true -period=30d -policy=kv_read
Key                  Value
---                  -----
token                hvs.CAESIHx-8ErMrbSoDNNJBeXxY6F3NkUi4d98PcPvhJqWKu4WGh4KHGh2cy5OeTRmb1dIaUw2dTd
mZFNkSFZGVmNDbXU
token_accessor       bXxhlrv44PGFQrtEZOUEgP1W
token_duration       720h
token_renewable      true
token_policies       ["default" "kv_read"]
identity_policies    []
policies             ["default" "kv_read"]
root@master-1:/mnt/data/vault/files# vault token lookup -accessor bXxhlrv44PGFQrtEZOUEgP1W
Key                 Value
---                 -----
accessor            bXxhlrv44PGFQrtEZOUEgP1W
creation_time       1782648750
creation_ttl        720h
display_name        token
entity_id           n/a
expire_time         2026-07-28T17:42:30.740427211+05:30
explicit_max_ttl    0s
id                  n/a
issue_time          2026-06-28T17:42:30.740432265+05:30
meta                <nil>
num_uses            0
orphan              false
path                auth/token/create
period              720h
policies            [default kv_read]
renewable           true
ttl                 719h59m46s
type                service
```

> when a token is created the policy is attached to grant capabilities defining what the token can do. And a default policy are granted to the token by default. if no policy is defined during token creation then the parent policies are applied.

what if a token is created without any passing any policy via root token ??

> the token created will be a new root token. with root policy attached

```
root@master-1:/mnt/data/vault/files# vault token create
Key                  Value
---                  -----
token                hvs.L3VXElXfT1C3VZKK1cdyt6Es
token_accessor       jEqVn4PUzrE5lAi3GRQU4Ggu
token_duration       ∞
token_renewable      false
token_policies       ["root"]
identity_policies    []
policies             ["root"]
root@master-1:/mnt/data/vault/files#
```

1. vault token used to perform action according to its capabilities

© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/mnt/data/vault/files# vault token capabilities -accessor bXxhlrv44PGFQrtEZOUEgP1W kv
create, list, read
```

1. vault token can be renewed if renewable

> the token can be renewed is periodic token are created until either its period is exceeded or the time to live is ended.

as you can see in the image below. after renewal the ttl got reset and last renewal time is added which tells when was the last time token ttl was renewed and expiry time is increased as the token is recently renewed.

```
root@master-1:/mnt/data/vault/files# vault token renew -accessor bXxhlrv44PGFQrtEZOUEgP1W
Key                  Value
---                  -----
token                n/a
token_accessor       bXxhlrv44PGFQrtEZOUEgP1W
token_duration       720h
token_renewable      true
token_policies       ["default" "kv_read"]
identity_policies    []
policies             ["default" "kv_read"]
root@master-1:/mnt/data/vault/files# vault token lookup -accessor bXxhlrv44PGFQrtEZOUEgP1W
Key                  Value
---                  -----
accessor             bXxhlrv44PGFQrtEZOUEgP1W
creation_time        1782648750
creation_ttl         720h
display_name         token
entity_id            n/a
expire_time          2026-07-28T17:42:52.559030235+05:30
explicit_max_ttl     0s
id                   n/a
issue_time           2026-06-28T17:42:30.740432265+05:30
last_renewal         2026-06-28T17:42:52.559030383+05:30
last_renewal_time    1782648772
meta                 <nil>
num_uses             0
orphan               false
path                 auth/token/create
period               720h
policies             [default kv_read]
renewable            true
ttl                  719h59m57s
type                 service
```

1. vault token revoked

> once token is revoked either manually or on its expiry it looses all capabilities and you can see is not usable anymore. vault throws an error if the revoked tokens are used.

© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/mnt/data/vault/files# vault token revoke -accessor bXxhlrv44PGFQrtEZOUEgP1W
Success! Revoked token (if it existed)
root@master-1:/mnt/data/vault/files# vault token lookup -accessor bXxhlrv44PGFQrtEZOUEgP1W
Error looking up token: Error making API request.

URL: POST https://192.168.29.10:8200/v1/auth/token/lookup-accessor
Code: 400. Errors:

* 1 error occurred:
        * invalid accessor


root@master-1:/mnt/data/vault/files# vault token capabilities -accessor bXxhlrv44PGFQrtEZOUEgP1W kv
Error listing capabilities: Error making API request.

URL: POST https://192.168.29.10:8200/v1/sys/capabilities-accessor
Code: 400. Errors:

* 1 error occurred:
        * invalid accessor
```


© 2026 v1 Sachin Kansal . All rights reserved.

Service Tokens vs Batch Tokens

Service and Batch tokens are main 2 types of tokens which are used by applications.

The major difference between these token types is whether they are stored in Vault's storage backend. By default any token is created it is of type service and they are stored in vault storage backend and access via accessor so whenever a request authenticated with a service token reaches Vault, Vault validates the token against its token store. Since service tokens are persisted in the storage backend, this validation involves reading the token entry maintained by Vault. This increase latency and creates pressure on vault. Here Batch token wins they are not stored in vault storage backend or any ram, so how do vault know is token is valid or not or which policies are applied to it ? Batch tokens are self-contained. They are cryptographically signed and contain the information required for Vault to validate them, such as policies, TTL, and other metadata. Therefore Vault does not need to perform a storage lookup to authenticate them and hence the size of these token are larger than service token they best fit for backend application or some ETL or Cronjob which require to access secret from vault more frequently. Not only that batch token are best suited for Disaster recovery and replication as there is no no need to reconfirm the token validity from primary vault process.

> Batch Token

```
master@master-1:~$ vault token create -ttl=50m -type batch -use-limit=5 -renewable=true -policy=kv_read
_create
Key                  Value
---                  -----
token                hvb.AAAAAQIVksjF-WL0zDfJ_2rgpBcSQTxV06NdTAyVl7-aANyZlH9ZZfsmzh1MyzrhFJichS-_xSftSHjnSbJGD0VvI
jFRpXEbi8FcD_HEqOneHwwSuFzEEyb-UXU4l32XKIJ6903vicCIgV0IdZbZE1T2_07Nton8ZUoHoMECNTr9lttUv8NXnMk
token_accessor       n/a
token_duration       50m
token_renewable      false
token_policies       ["default" "kv_read_create"]
identity_policies    []
policies             ["default" "kv_read_create"]
```

as you can see there is no accessor because they are not stored in backend.

they are not renewable as vault does not track them so it cannot increase their leases.

> Service Token

```
master@master-1:~$ vault token create -ttl=50m -use-limit=5 -renewable=true -policy=kv_read_create
Key                  Value
---                  -----
token                hvs.CAESILUHovB5Cmk_0e9hwY1E_crIXqplPUuwW1simz91FiWiGh4KHGh2cy43Vzl1aUtOU2NiRXI4NjBWRGJkTHlPS
zM
token_accessor       k5X0jm0X0buF72RKnTW9ksfG
token_duration       50m
token_renewable      true
token_policies       ["default" "kv_read_create"]
identity_policies    []
policies             ["default" "kv_read_create"]
```

Because service token are stored in storage backend they have an accessor attached to it. this accessor is later used to access the token information aka lookup or revoke the token without revealing its value.

Note this token is tracked so it can be renewed granted done before expiry.

although these token are granted a duration and made periodic in nature they will expire after 5 uses because use-limit is assigned

limit token usage

> use limit

when an administration task is done and a token is shared with a team to run known/counted number of operations it is advised to use-limit as it auto revokes the tokens once the limit is exhausted.

© 2026 v1 Sachin Kansal . All rights reserved.

num_uses follow the following function

1. N>0 i.e. number of times token can be used
2. -1, 0 i.e. indicating the token created has no usage operational limit and can be used unlimited times until the lease is revoked.

> This is how accessor is used to get information about the specific token.

```
master@master-1:~$ vault token lookup -accessor k5X0jm0X0buF72RKnTW9ksfG
Key                 Value
---                 -----
accessor            k5X0jm0X0buF72RKnTW9ksfG
creation_time       1782591280
creation_ttl        50m
display_name        token
entity_id           n/a
expire_time         2026-06-28T02:34:42.6202707+05:30
explicit_max_ttl    0s
id                  n/a
issue_time          2026-06-28T01:44:42.620277478+05:30
meta                <nil>
num_uses            5
orphan              false
path                auth/token/create
policies            [default kv_read_create]
renewable           true
ttl                 48m41s
type                service
```


© 2026 v1 Sachin Kansal . All rights reserved.

Root token

Root tokens are the token with root privileges in the vault anyone with the root token is free to do whatever he/she/it want to do on the vault. delete any secret disable a authentication method or secret engine, seal the vault and so on. also these token never expire so as lon as it lives the vault is vulnerable and prone to compromised.

hence the root token should be revoked and recovered when it is required.

Re-generating the root token

What we are aiming for is that there are no active token with root privileges but we do need a token which can generate a root token in case of crisis.

A policy with minimal access to generate a root token.

```
path "sys/generate-root/attempt" {
  capabilities = ["create", "update", "read", "sudo"]
}

path "sys/generate-root/update" {
  capabilities = ["create", "update", "read", "sudo"]
}

path "sys/generate-root/attempt/*" {
  capabilities = ["read", "sudo"]
}

path "sys/generate-root/status" {
  capabilities = ["read", "sudo"]
}
```

```
root@master-1:/mnt/data/vault/files# vault policy write root-maker ./root-maker-policy.hcl
Success! Uploaded policy: root-maker
root@master-1:/mnt/data/vault/files# vault token create -policy root-maker
Key                  Value
---                  -----
token                hvs.CAESIOoMX0c7GmjfngAmI7dII6TnGIWCg7w1OkMrZV6UtFAEGh4KHGh2cy5PSTdLeWlDSjdqWjVR
OUVKcUh3a1dpZ2Y
token_accessor       APFwStUaJJKznUCo0ZwSZpX2
token_duration       768h
token_renewable      true
token_policies       ["default" "root-maker"]
identity_policies    []
policies             ["default" "root-maker"]
```

> login with new token
>
> ```
> vault login
> ```


© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/mnt/data/vault/files# export VAULT_TOKEN=""
root@master-1:/mnt/data/vault/files# vault login
Token (will be hidden):
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.

Key                  Value
---                  -----
token                hvs.CAESIOoMX0c7GmjfngAmI7dII6TnGIWCg7w1OkMrZV6UtFAEGh4KHGh2cy5PSTdLeWlDSjdqWjVR
OUVKcUh3a1dpZ2Y
token_accessor       APFwStUaJJKznUCo0ZwSZpX2
token_duration       767h59m20s
token_renewable      true
token_policies       ["default" "root-maker"]
identity_policies    []
policies             ["default" "root-maker"]
```

Root token generation

© 2026 v1 Sachin Kansal . All rights reserved.

1. start the process of generating the root

```
vault operator generate-root -init
```

take a note of otp it will be required to decode the generated token

2. pass the seal keys

```
vault operator generate-root
```

once completed a encoded token is generated share it with the person having otp generated in first step.

3. use otp to decode the token

```
vault operator generate-root -otp=<otp> -decode=<encoded_token>
```

```
root@master-1:/mnt/data/vault/files# vault operator generate-root -init
A One-Time-Password has been generated for you and is shown in the OTP field.
You will need this value to decode the resulting root token, so keep it safe.
Nonce         b592c067-43ff-6900-ccec-4006e7f2f6cc
Started       true
Progress      0/3
Complete      false
OTP           EOnITYaXkD99DbgJo3Xh4k6vvNGp
OTP Length    28
root@master-1:/mnt/data/vault/files# vault operator generate-root
Operation nonce: b592c067-43ff-6900-ccec-4006e7f2f6cc
Unseal Key (will be hidden):
Nonce       b592c067-43ff-6900-ccec-4006e7f2f6cc
Started     true
Progress    1/3
Complete    false
root@master-1:/mnt/data/vault/files# vault operator generate-root
Operation nonce: b592c067-43ff-6900-ccec-4006e7f2f6cc
Unseal Key (will be hidden):
Nonce       b592c067-43ff-6900-ccec-4006e7f2f6cc
Started     true
Progress    2/3
Complete    false
root@master-1:/mnt/data/vault/files# vault operator generate-root
Operation nonce: b592c067-43ff-6900-ccec-4006e7f2f6cc
Unseal Key (will be hidden):
Nonce            b592c067-43ff-6900-ccec-4006e7f2f6cc
Started          true
Progress         3/3
Complete         true
Encoded Token    LTkdZzsgGRA+K3tVATg0JV52AVpNU1M9OywGAQ
root@master-1:/mnt/data/vault/files# vault operator generate-root -otp=EOnITYaXkD99DbgJo3Xh4k6vvNGp -decode=LTkdZzsgGR
A+K3tVATg0JV52AVpNU1M9OywGAQ
hvs.oyxHUoBlEZSo1EY2y8eKMbAq
```

4. Revoke the root

once crisis is completed revoke the root

```
vault token revoke <token>
```

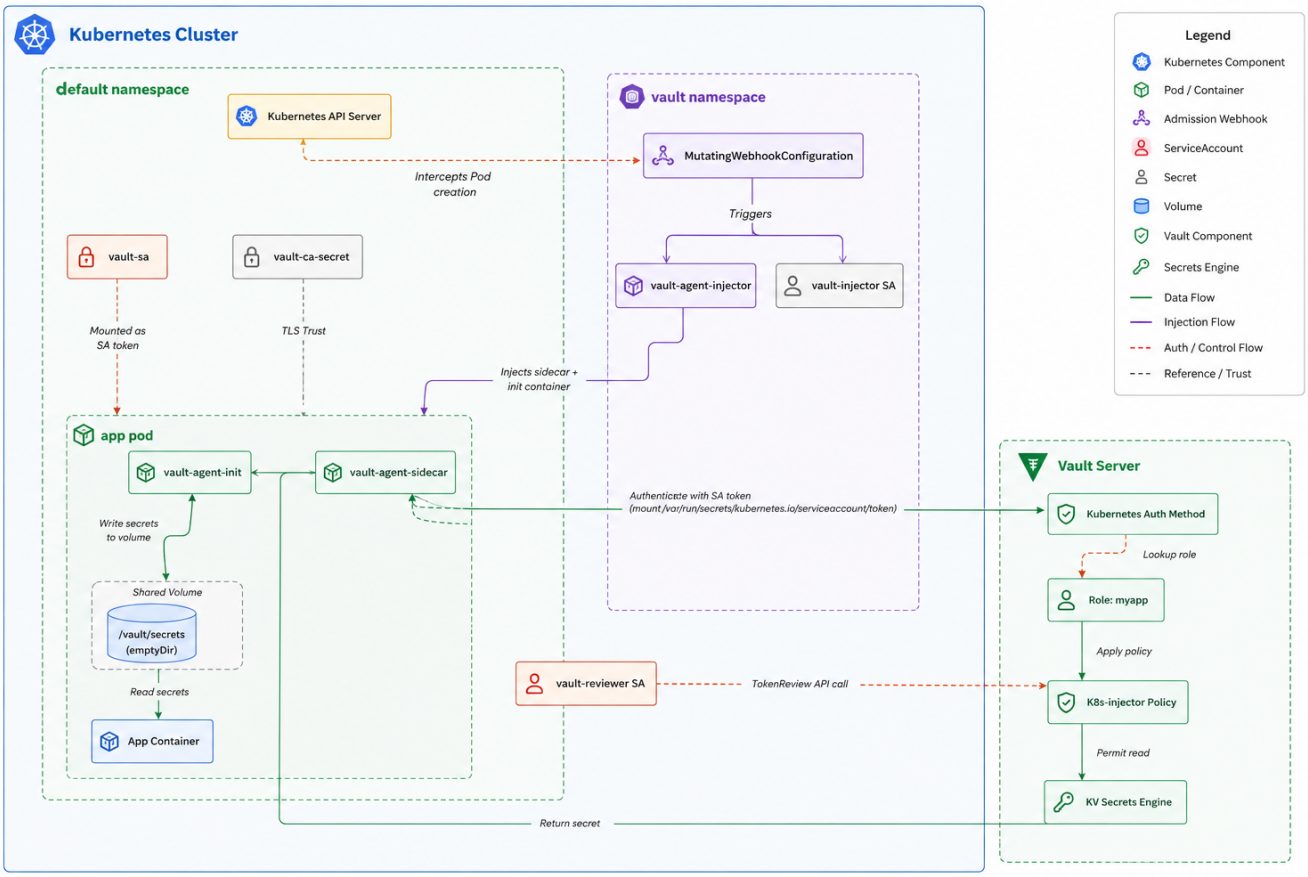
```
root@master-1:/mnt/data/vault/files# vault token revoke hvs.oyxHUoBlEZSo1EY2y8eKMbAq
Success! Revoked token (if it existed)
root@master-1:/mnt/data/vault/files#
```


© 2026 v1 Sachin Kansal . All rights reserved.

4. Auth Methods

Kubernetes authentication agent injector

Architecture of what we are building



Create secret with tls certficate used to validate vault

kubectl create secret generic vault-ca --from-file=ca.crt=/etc/vault.d/vault-cert.pem

Install the agent injector from helm

> to add the hashicorp official charts

```
helm repo add hashicorp https://helm.releases.hashicorp.com
```

> to get a local pull and study the values file

```
helm pull hashicorp/vault
```

> install agent injector with values files

```
helm install vault-injector hashicorp/vault --namespace vault --values ./values.yaml
```

refrence values.yaml is defined in `.\agent_injector_manifests\values.yaml`

© 2026 v1 Sachin Kansal . All rights reserved.

Create reviewer service account and token

> This service account is used to login into the vault and verify the connection
>
> ```
> kubectl create serviceaccount vault-reviewer
>
> REVIEWER=$(kubectl get secret vault-reviewer-token -o jsonpath='{.data.token}' | base64 -d)
> ```
>
> this Reviewer token will be used in kubernetese auth config

Note: If not done properly 403 permission denied errors will pop in vault-agent-init

Enable kubernetes authentication mechanism

```
vault auth enable kubernetes

vault write auth/kubernetes/config \
  kubernetes_host="https://192.168.29.10:6443" \
  kubernetes_ca_cert=@/mnt/data/k3s/server/tls/server-ca.crt \
  token_reviewer_jwt="$REVIEWER" \
  disable_local_ca_jwt="true"
```

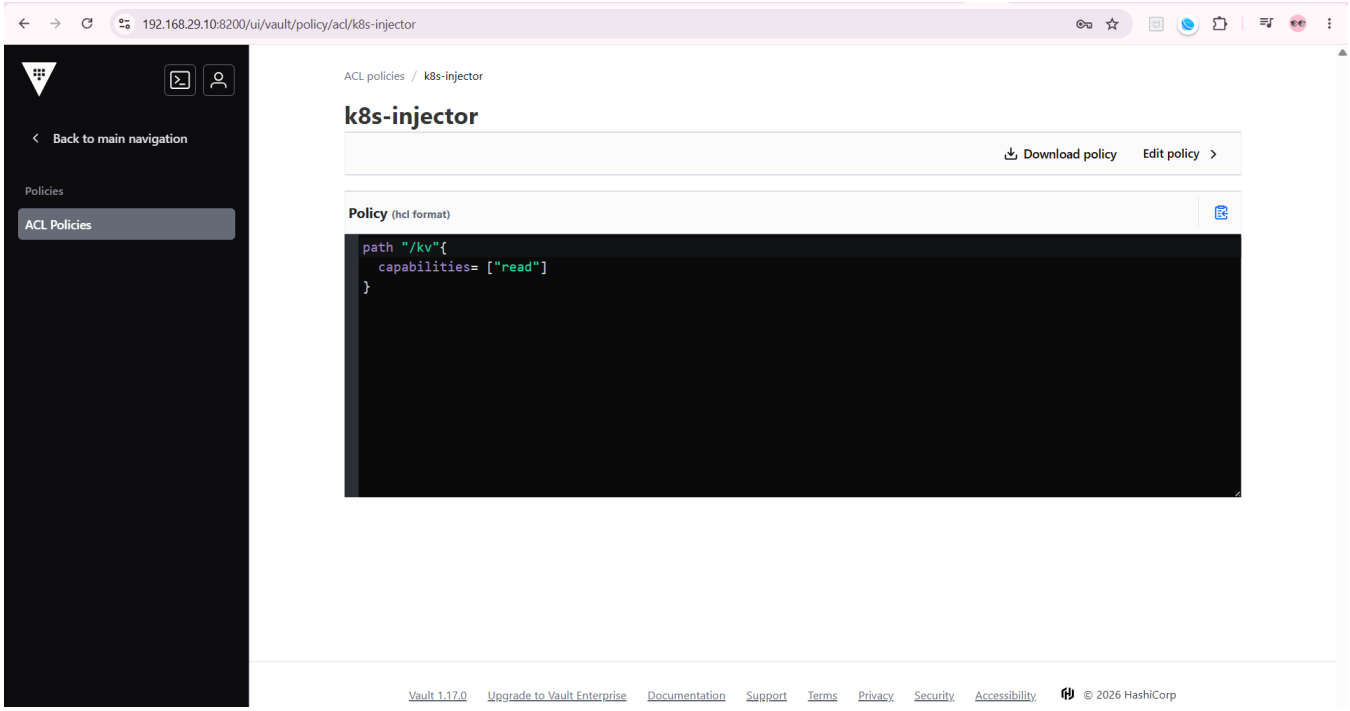
> disable_local_ca_jwt, tells vault it is not in k8s cluster and should not look for local ca and jwt. thats why we are providing the serviceaccount jwt and and k8s apiserver Certificate to authenticate and establish connection between vault and k8s.

```
root@master-1:/mnt/data/vault# vault auth enable kubernetes
Success! Enabled kubernetes auth method at: kubernetes/
root@master-1:/mnt/data/vault# vault auth list
Path           Type          Accessor                    Description                Version
----           ----          --------                    -----------                -------
kubernetes/    kubernetes    auth_kubernetes_781c64f9    n/a                        n/a
token/         token         auth_token_4e3c59fb         token based credentials    n/a
userpass/      userpass      auth_userpass_25d9dc42      n/a                        n/a
root@master-1:/mnt/data/vault#
```


© 2026 v1 Sachin Kansal . All rights reserved.

Create role and policy

policy

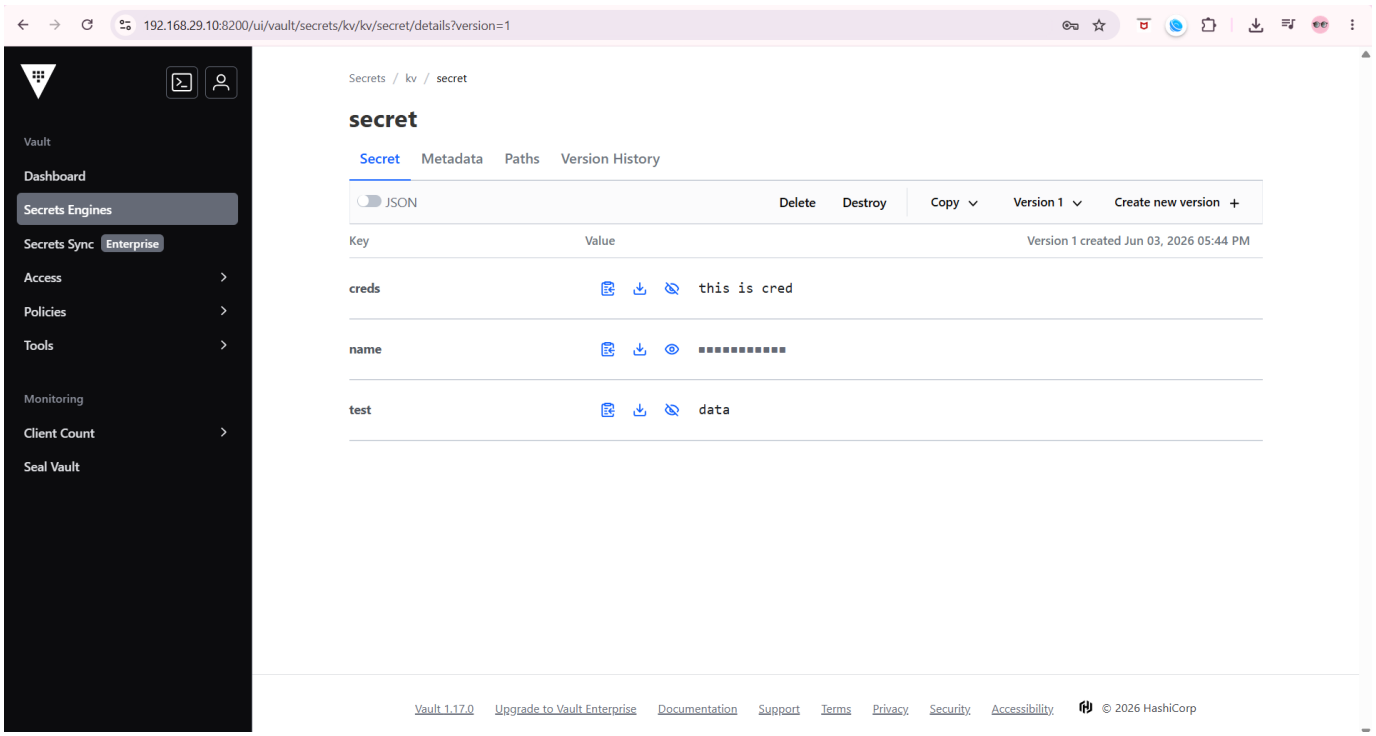


Role:

```
vault write auth/kubernetes/role/myapp \
  bound_service_account_names=vault-sa \
  bound_service_account_namespaces=default \
  policies=k8s-injector \
  ttl=1h \
  audience="k3s" \
  alias_name_source="serviceaccount_name"
```


© 2026 v1 Sachin Kansal . All rights reserved.

Create secret in vault



Create a test app and its service account

> use deployment.yam to create a deployment

Important annotations

```
      annotations:
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/role: "myapp"
        vault.hashicorp.com/agent-inject-secret-mysecretfile: "kv/data/secret"
        vault.hashicorp.com/tls-secret: "vault-ca"
        vault.hashicorp.com/ca-cert: "/vault/tls/ca.crt"
```

```
root@master-1:/mnt/data/vault# kubectl logs po/vault-app-b65c4b998-bxjfn
Defaulted container "busybox" out of: busybox, vault-agent, vault-agent-init (init)
data: map[creds:this is cred name:sachin test:data]
metadata: map[created_time:2026-06-03T12:14:57.661482898Z custom_metadata:<nil> deletion_time: destroyed:false version:1]
```

Agent_injector_manifest

- values.yaml used in helm install values.yaml
- deployment.yaml deployment.yaml
- token_reviewer
  - create serice_account using `kubectl create sa <serviceaccount_name> -n <namespace>`
  - Role tokenreviewer_role
  - Secret token_Secret

© 2026 v1 Sachin Kansal . All rights reserved.

UserPass authentication

> Till now we were using the token for authentication on vault which is default mechanism but this is not recommended for users as no one gonna remember these tokens so we will use username and password.

vault auth help

to see basic auth commands i.e. - enable - disable - help - list - move - tune

```
vault auth --help
```

check which auth methods are enable

```
vault auth list
```

Output:

```
Path      Type     Accessor               Description                Version
----      ----     --------               -----------                -------
token/    token    auth_token_4e3c59fb    token based credentials    n/a
```

to see basic usage of authentication method

```
vault auth  help userpass
```

Output:

```
Usage: vault login -method=userpass [CONFIG K=V...]

  The userpass auth method allows users to authenticate using Vault's
  internal user database.

  Authenticate as "sally":

      $ vault login -method=userpass username=sally
      Password (will be hidden):

  Authenticate as "bob":

      $ vault login -method=userpass username=bob password=password

Configuration:

  password=<string>
      Password to use for authentication. If not provided, the CLI will prompt
      for this on stdin.

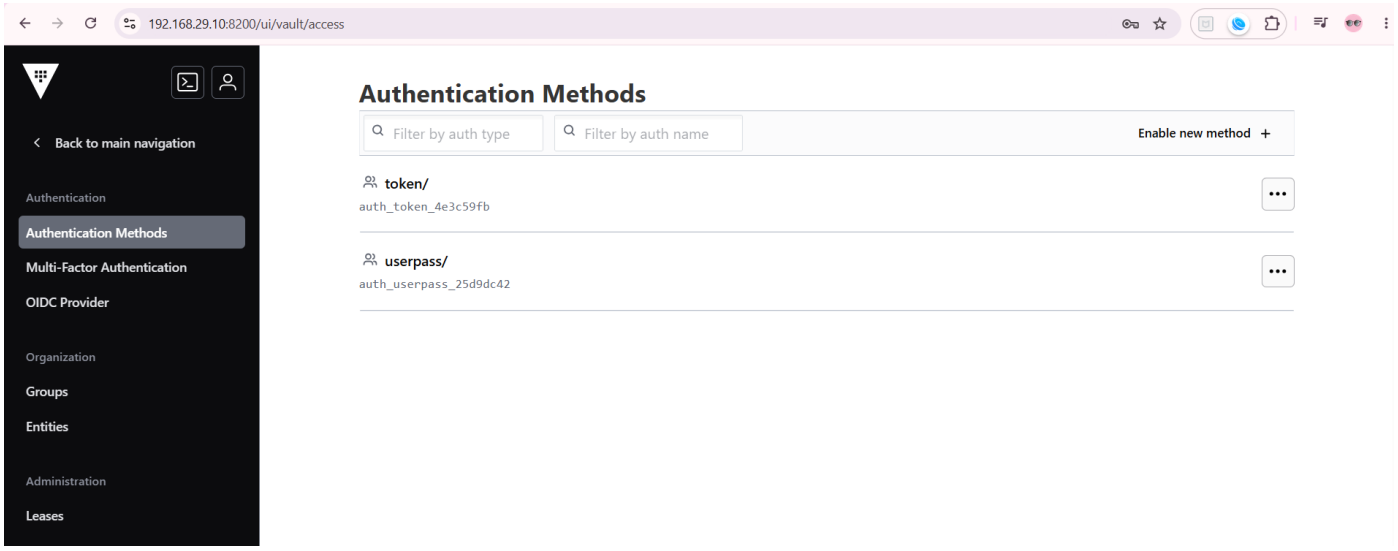
  username=<string>
      Username to use for authentication.
```

enable the authentication method

as we have seen only token authentication method is enabled till now

```
vault auth enable userpass
```


© 2026 v1 Sachin Kansal . All rights reserved.




```
Success! Enabled userpass auth method at: userpass/
```

> now auth method is enabled

```
vault auth list
```

Output:

```
Path         Type        Accessor                 Description                Version
----         ----        --------                 -----------                -------
token/       token       auth_token_4e3c59fb      token based credentials    n/a
userpass/    userpass    auth_userpass_25d9dc42   n/a                        n/a
```

creating the user

before creating the user create the acl policy that user needs to be associated. this make sure the user created has least privilage or minimal access.

Default acl policy is attached to all by default.

```
vault write auth/userpass/users/sachin password="pass@123" policies="kv-engine"
```

Output:

```
Success! Data written to: auth/userpass/users/sachin
```

user Login

exported VAULT_TOKEN takes precedince we were working on cli using root till now even if we do login it will not impact as root token will be used for request. so unset the value first.

```
WARNING! The VAULT_TOKEN environment variable is set! The value of this
variable will take precedence; if this is unwanted please unset VAULT_TOKEN or
update its value accordingly.
```

```
export VAULT_TOKEN=""
```

TO VERIFY THAT ROOT IS LOGGED OUT ACTIVE

```
vault read /auth/userpass/users/sachin
```

Output:

```
Error reading auth/userpass/users/sachin: Error making API request.

URL: GET https://192.168.29.10:8200/v1/auth/userpass/users/sachin
Code: 403. Errors:
```


© 2026 v1 Sachin Kansal . All rights reserved.

```
* 1 error occurred:
        * permission denied
```

```
vault login -method=userpass username=sachin
Password (will be hidden):
```

Output:

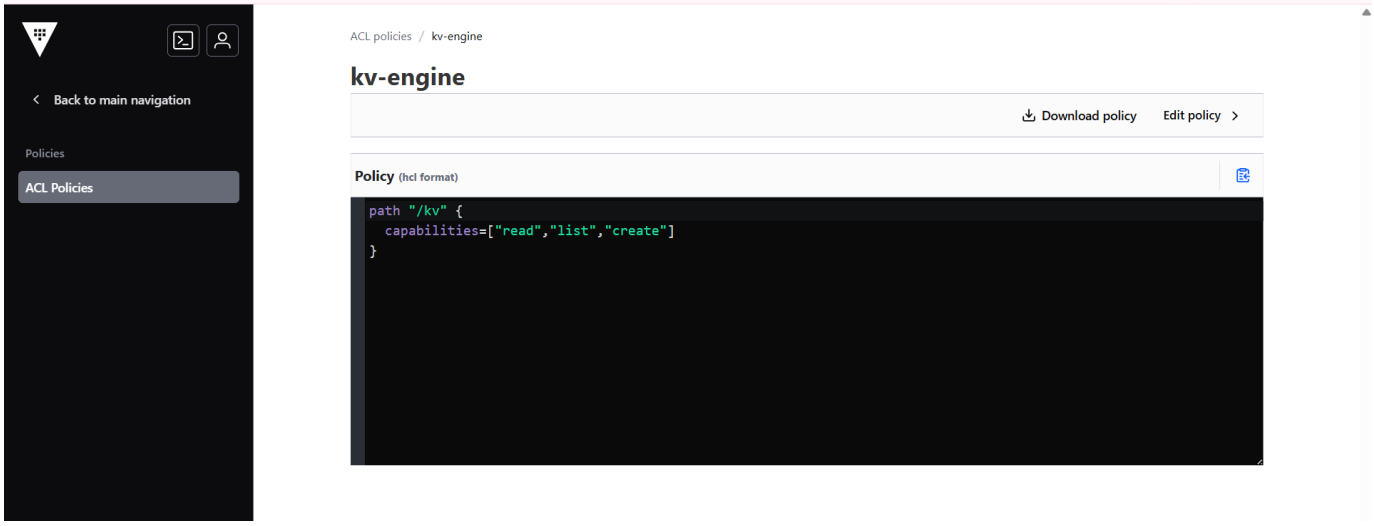
```
Success! You are now authenticated. The token information displayed below
is already stored in the token helper. You do NOT need to run "vault login"
again. Future Vault requests will automatically use this token.
Key                    Value
---                    -----
token                  hvs.CAExxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
token_accessor         n4TK5lzTXciWLbB41mzXwyFz
token_duration         768h
token_renewable        true
token_policies         ["default" "kv-engine"]
identity_policies      []
policies               ["default" "kv-engine"]
token_meta_username    sachin
```

> A temperory token is created and registered by default internal all authentication methods generate a token to authenticate but these are shortlived and session based. on each login new temporary token is created. even when logged in via UI
>
> this token gets stored at
>
> ```
> cat ~/.vault-token
> hvs.CAExxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
> ```
>
> until logged out.

limited user access as per the policy

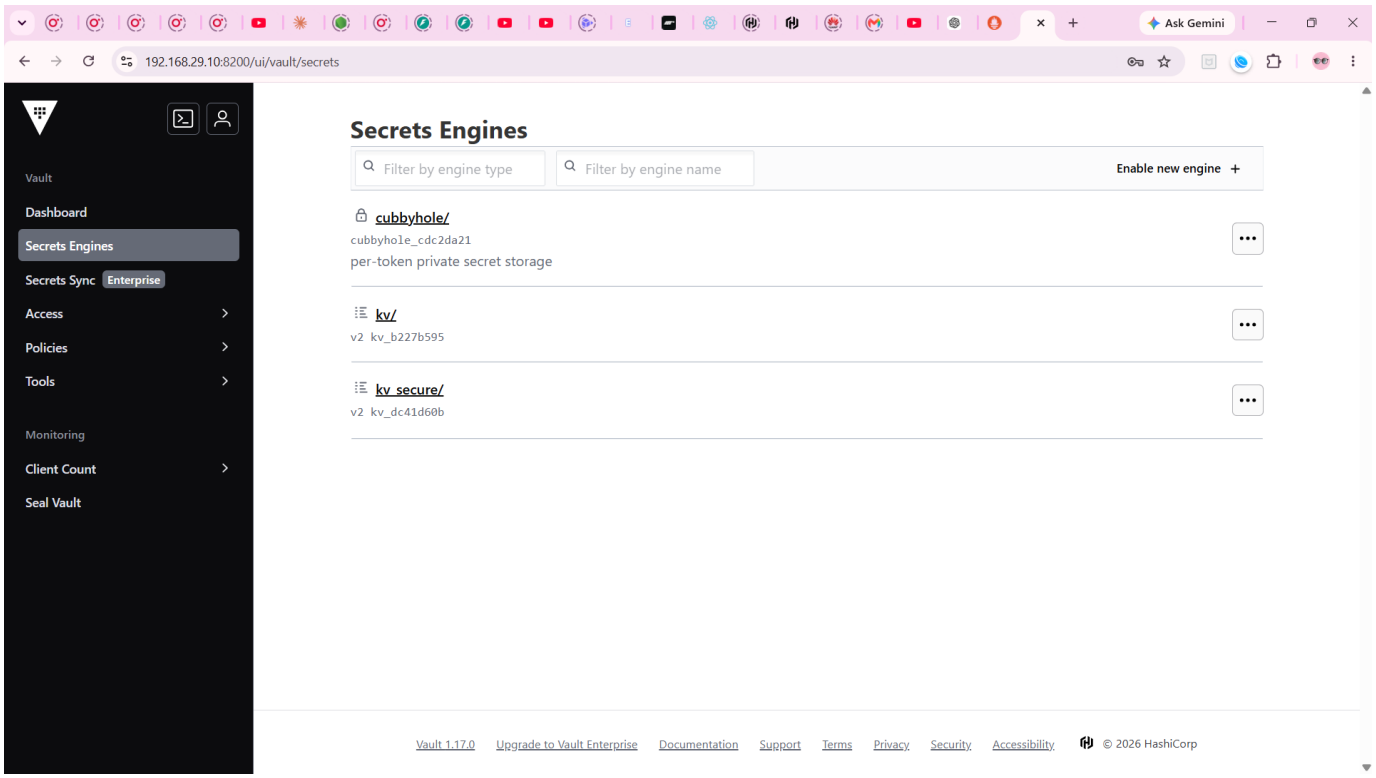
> user get access to /kv path based on the policy attached not kv_secure whereas root can see it all.

policy

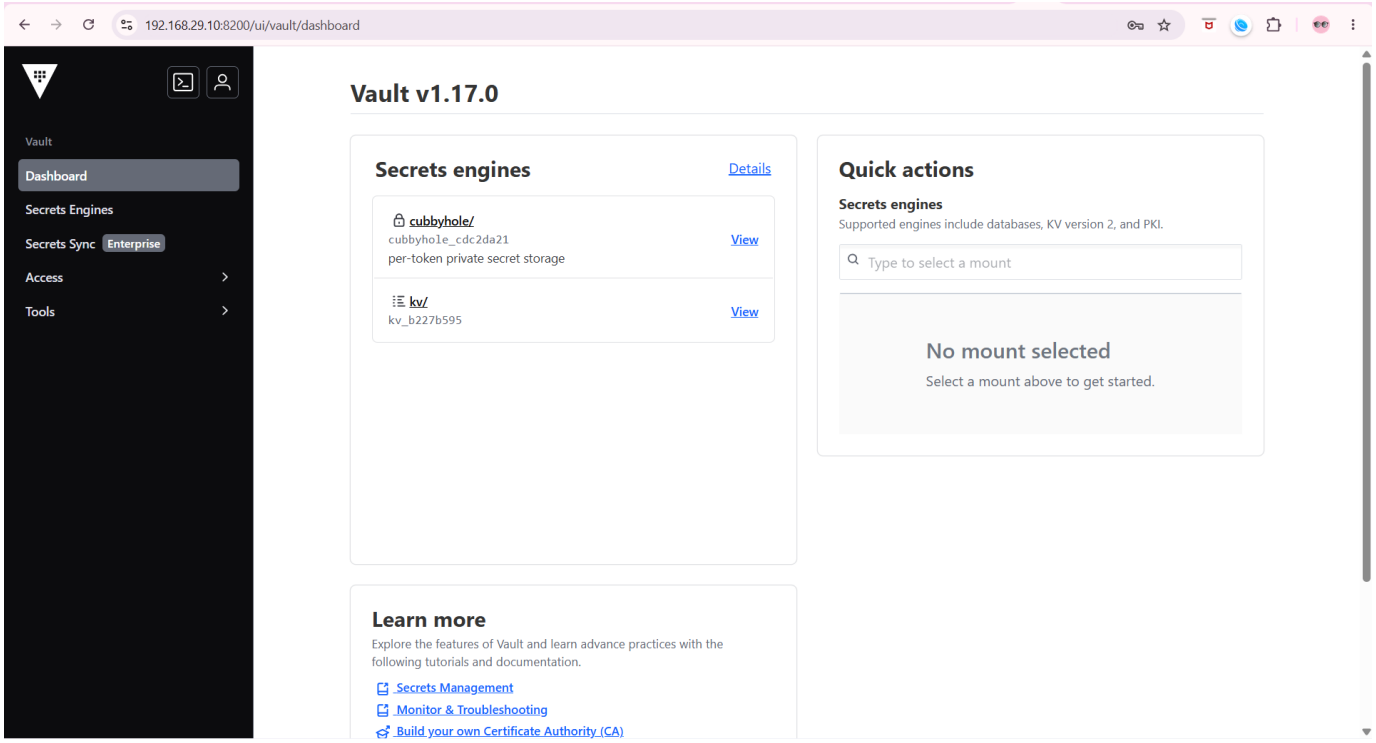


© 2026 v1 Sachin Kansal . All rights reserved.

root access



user access



© 2026 v1 Sachin Kansal . All rights reserved.

5. Secrets Engines

Key/Value pair

Most Common and simplest Secret Engine used mostly in every organisation

- Save information in the key value pair
- data saved in json format

Enable engine

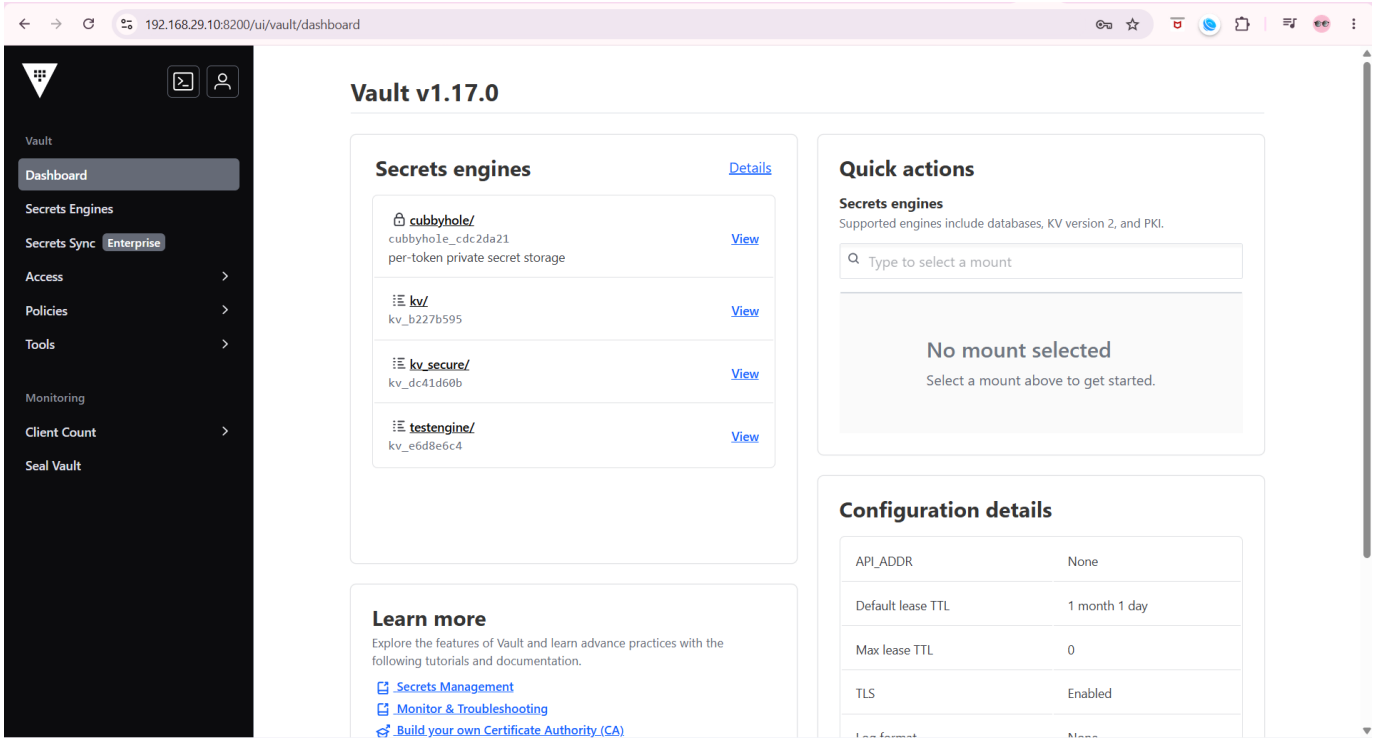
> kv secret engine comes in 2 different version 1 and 2. version 2 is recommended and we will e using the same.

```
vault secrets enable -version=2 -path=testengine kv
```

> path is the important configuration as it define the kv secret engine name and root path which allow us to create ultiple kv secret engines.
>
> Default value of path is kv

```
root@master-1:/home/master# vault secrets enable -version=2 -path=testengine kv
Success! Enabled the kv secrets engine at: testengine/
root@master-1:/home/master#
```

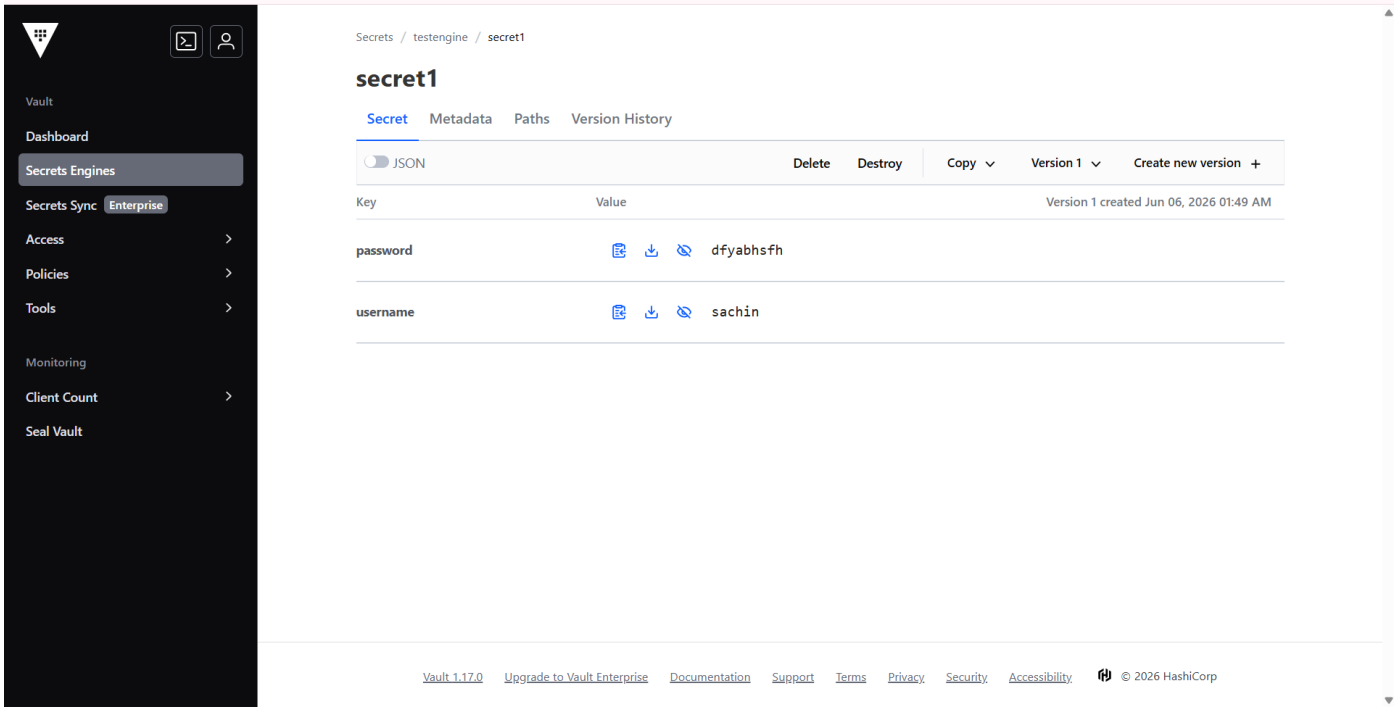
UI



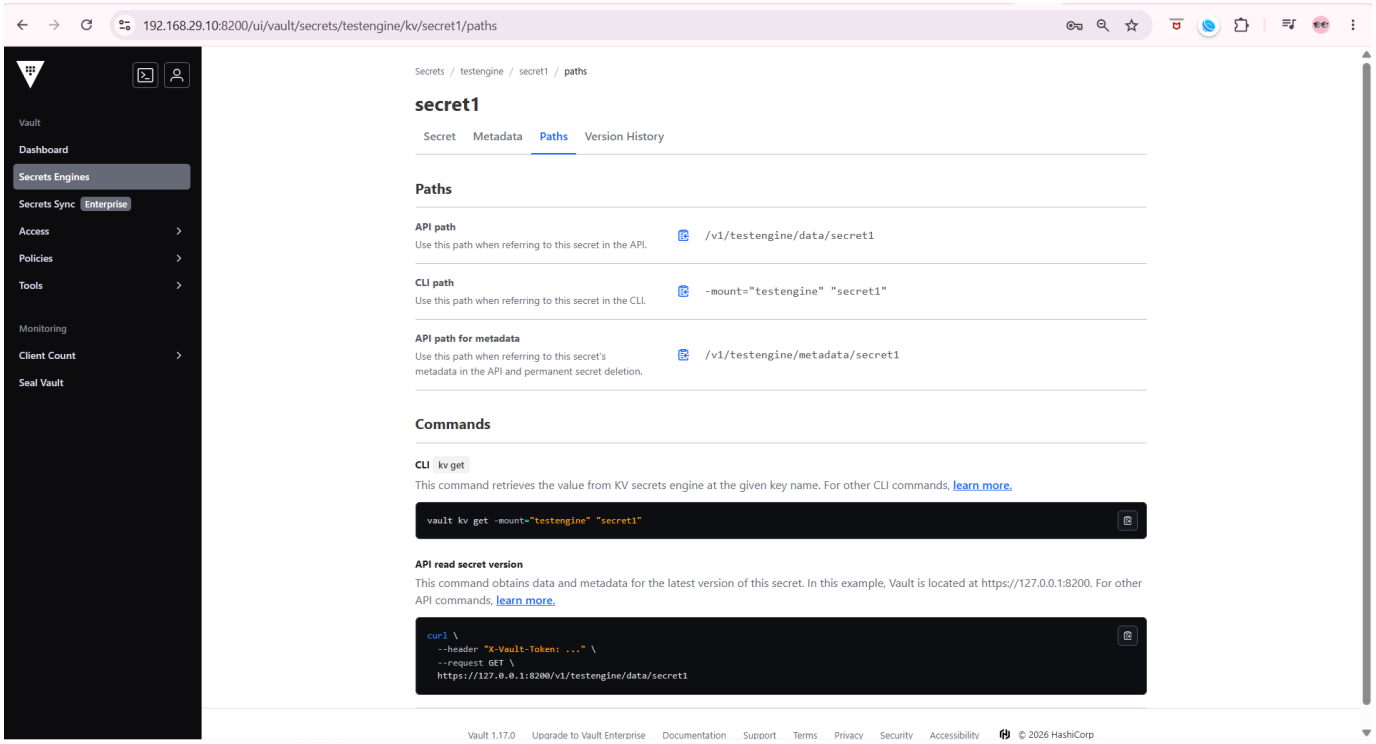
creating secret

```
vault kv put -mount=testengine secret1 username=sachin password=dfyabhsfh
```


© 2026 v1 Sachin Kansal . All rights reserved.



> **To Remember:** the path of secrets vary with the tool used for access.



versioning

> kv version2 comes with versioning

As you can see in the image new version 2 of the same secret is created

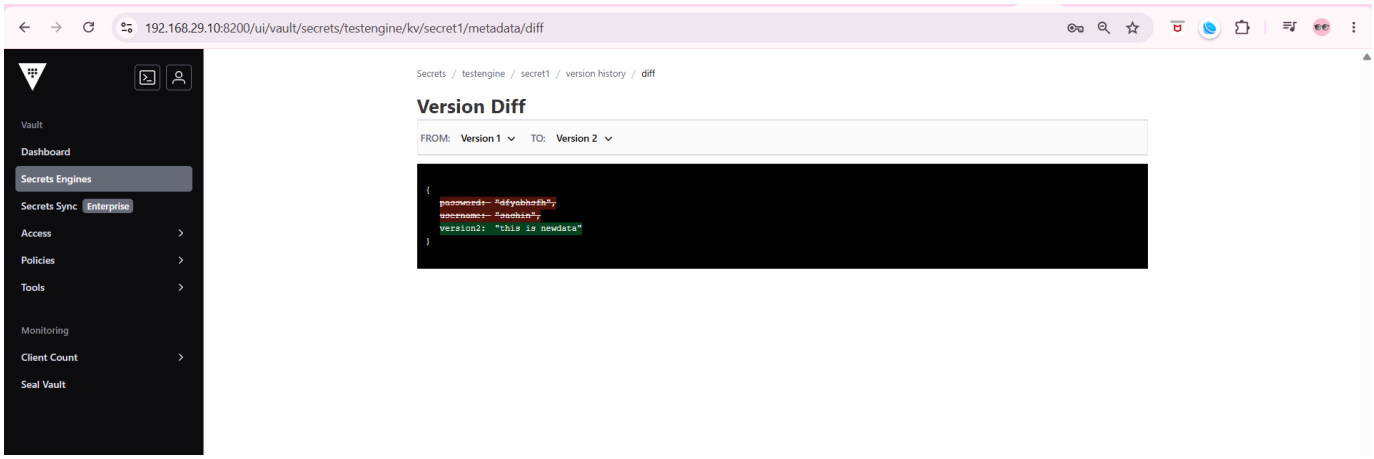
© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/home/master# vault kv put -mount=testengine secret1 username=sachin password=dfyabhsfh
===== Secret Path =====
testengine/data/secret1

======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T20:19:13.856863264Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            1
root@master-1:/home/master# vault kv put -mount=testengine secret1 version2="this is newdata"
===== Secret Path =====
testengine/data/secret1

======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T20:23:05.563234324Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            2
root@master-1:/home/master#
```

> in ui you can see the changes made

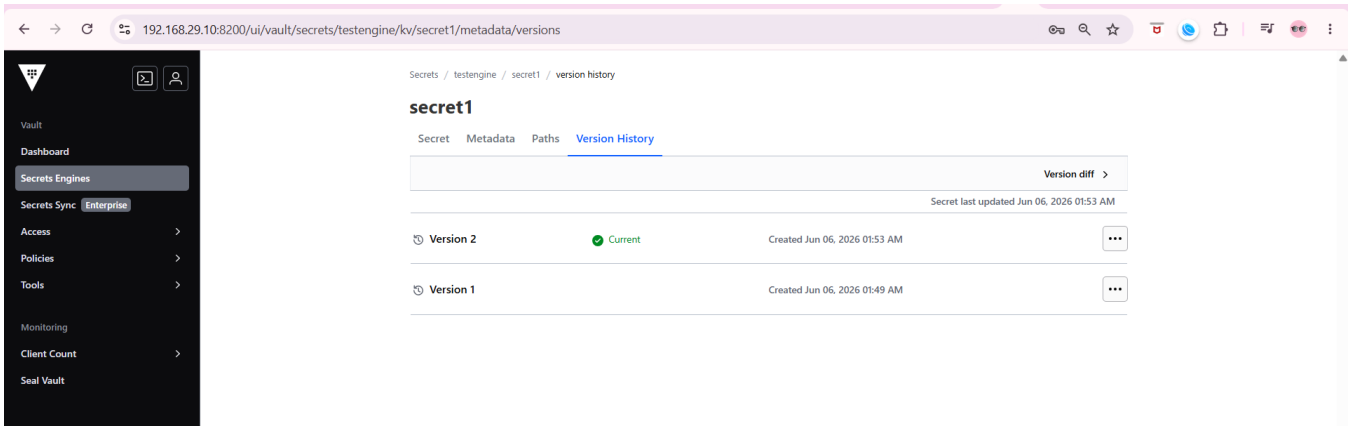


rollback the secret version

> **To Remember:**even if the secret is rollback a new version of that secret will be created having the data copied from previous version.

© 2026 v1 Sachin Kansal . All rights reserved.

for instance if we rollback from v5 to v3 then a new v6 will be created having the data stored in v3

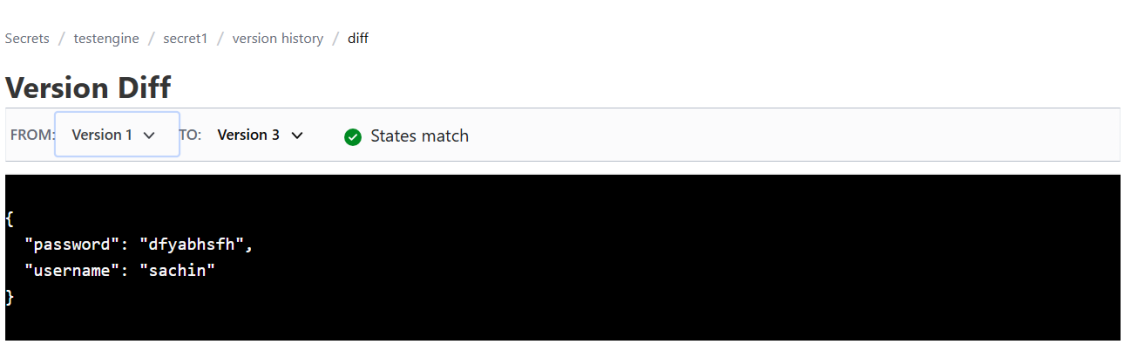
> current active version
>
> 
>

```
vault kv rollback -mount=testengine -version=1 secret1
```

```
root@master-1:/home/master# vault kv rollback -mount=testengine -version=1 secret1
Key                 Value
---                 -----
created_time        2026-06-05T20:36:30.001999761Z
custom_metadata     <nil>
deletion_time       n/a
destroyed           false
version             3
root@master-1:/home/master#
```

Ln 52, Col 1 Spaces: 4 UTF-8 CRLF { } Markdown Go Live

a new version 3 is created if compared with version1 it will be same.

> 
>

patching a secret

why patch??

> when we ran the put command it created a new verion(v2) of secret containing only the details passed from cli and it deleted the data stored in old version(v1) but that is not the requirement all the time. it could be that i want to edit only 1 kv pair or add a new kv pair without loosing existing data.
>
> Key is unique and used to identify whether it needs to update the value or add a new pair
>
> every update creates a new version.

© 2026 v1 Sachin Kansal . All rights reserved.

ADDING NEW PAIR

```
vault kv patch -mount=testengine secret1 patching="adding something new"
```

```
root@master-1:/home/master# vault kv patch -mount=testengine secret1 patching="adding something n
ew"
===== Secret Path =====
testengine/data/secret1

======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T20:57:43.310293447Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            4
root@master-1:/home/master#
```

Secrets / testengine / secret1

secret1

Secret | Metadata | Paths | Version History

JSON | Delete | Destroy | Copy | Version 4 | Create new version +

Key	Value	Version 4 created Jun 06, 2026 02:27 AM
password		dfyabhsfh
patching		adding something new
username		sachin

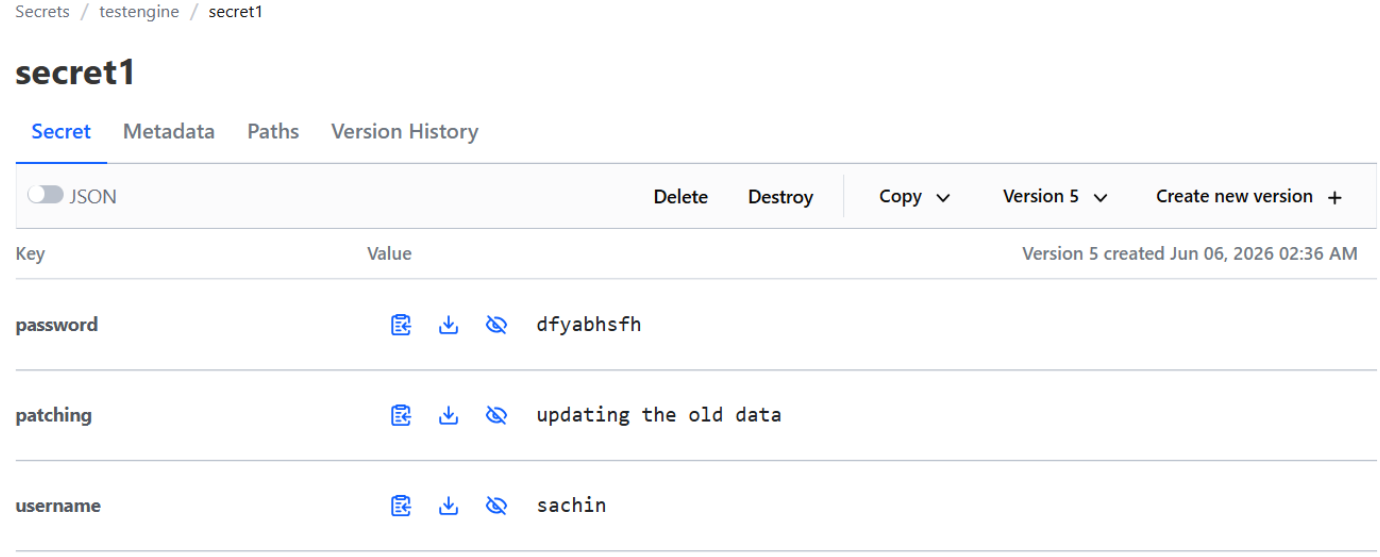
UPDATING A KV PAIR

```
vault kv patch -mount=testengine secret1 patching="updating the old data"
```


© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/home/master# vault kv patch -mount=testengine secret1 patching="updating the old d
ata"
===== Secret Path =====
testengine/data/secret1

======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T21:06:20.887628935Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            5
root@master-1:/home/master#
```



Deleting vs Destroying

Deleting	Destroying
only marks the secret version as deleted	permenantly deletes the version of secret
can be restored using undelete	version is lost form the chain forever no recovery possible

Deleting

we will see in the UI and cli using get command the version3 of secret name secret1 mounted at testengine gets marked deleted and becomes unreadable after deletion. later this will be recoverable using undelete.

```
vault kv delete -mount=testengine -versions=3  secret1
```


© 2026 v1 Sachin Kansal . All rights reserved.

Cli

```
root@master-1:/home/master# vault kv get -mount="testengine" -version=3  "secret1"
===== Secret Path =====
testengine/data/secret1

======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T20:36:30.001999761Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            3

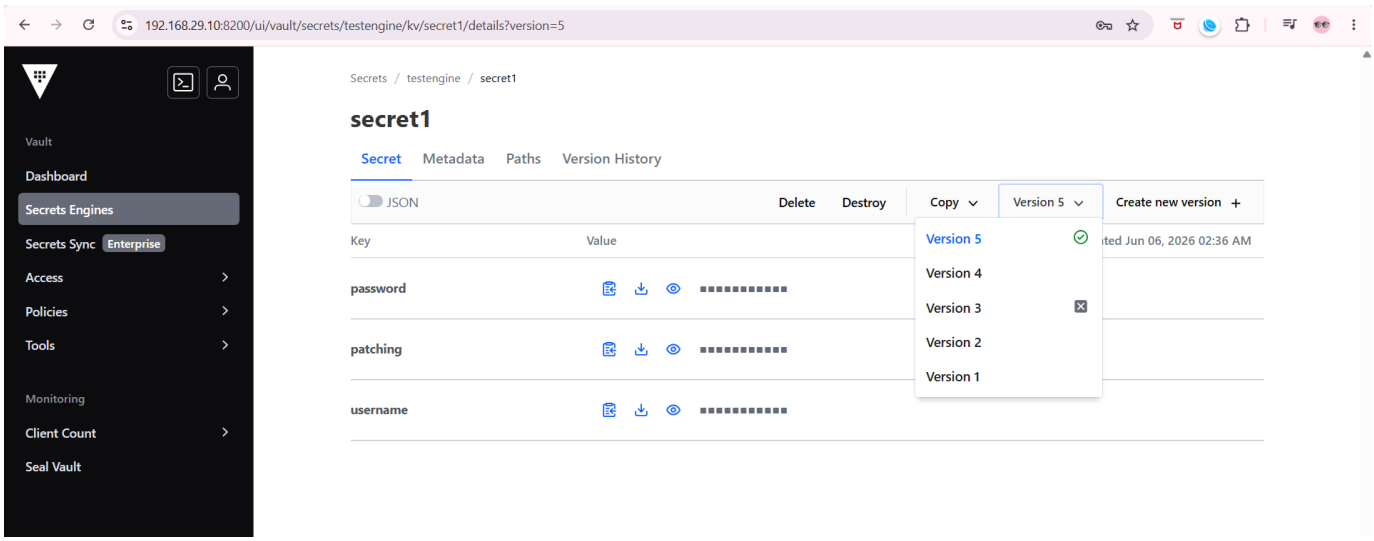
====== Data ======
Key         Value
---         -----
password    dfyabhsfh
username    sachin
root@master-1:/home/master# vault kv delete -mount=testengine -versions=3  secret1
Success! Data deleted (if it existed) at: testengine/data/secret1
root@master-1:/home/master# vault kv get -mount="testengine" -version=3  "secret1"
===== Secret Path =====
testengine/data/secret1

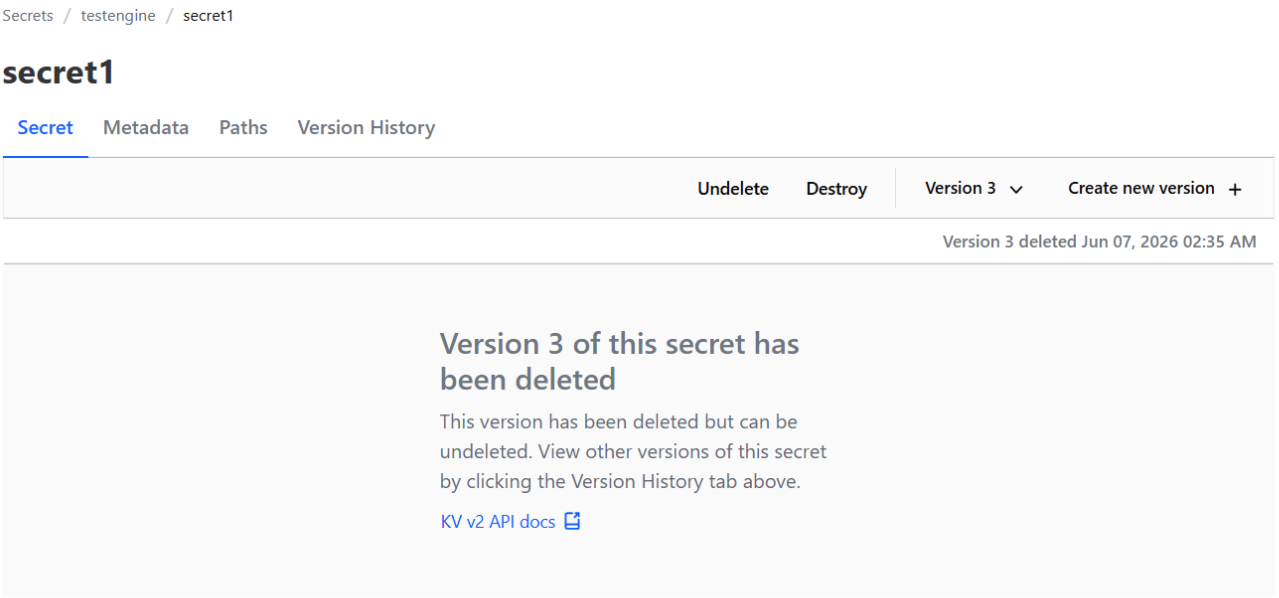
======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T20:36:30.001999761Z
custom_metadata    <nil>
deletion_time      2026-06-06T21:05:17.876682227Z
destroyed          false
version            3

root@master-1:/home/master#
```


© 2026 v1 Sachin Kansal . All rights reserved.

UI





Recovering the Lost version

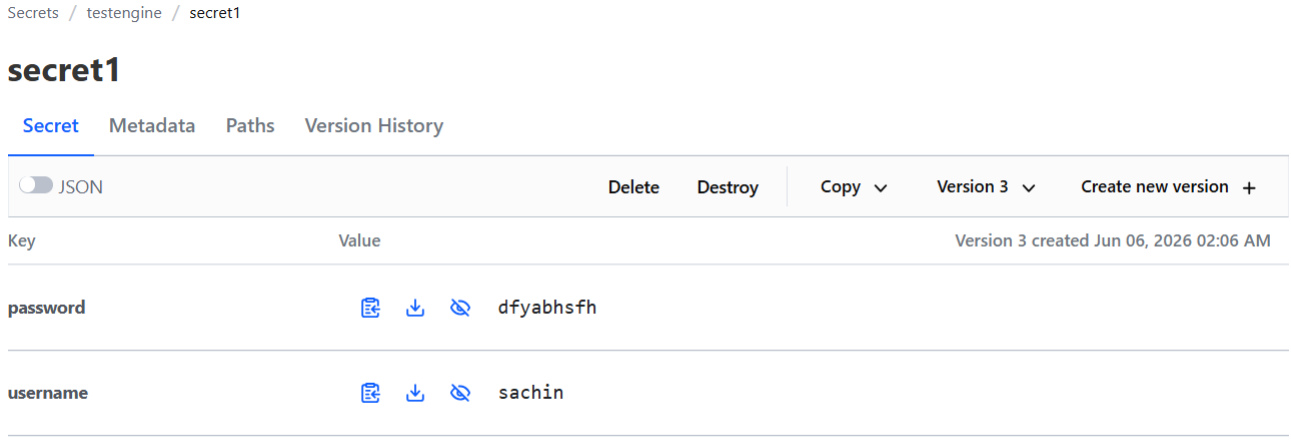
```
vault kv undelete -mount=testengine -versions=3  secret1
```


© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/home/master# vault kv undelete -mount=testengine -versions=3  secret1
Success! Data written to: testengine/undelete/secret1
root@master-1:/home/master# vault kv get -mount="testengine" -version=3  "secret1"
===== Secret Path =====
testengine/data/secret1

======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T20:36:30.001999761Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            3

====== Data ======
Key         Value
---         -----
password    dfyabhsfh
username    sachin
root@master-1:/home/master#
```



Now the lost data is recovered and deleted mark is removed. It can be usefull when we want to secure the data for some period of time without loosing it. thoose could be security patch, compliance tests moving the vault backened.

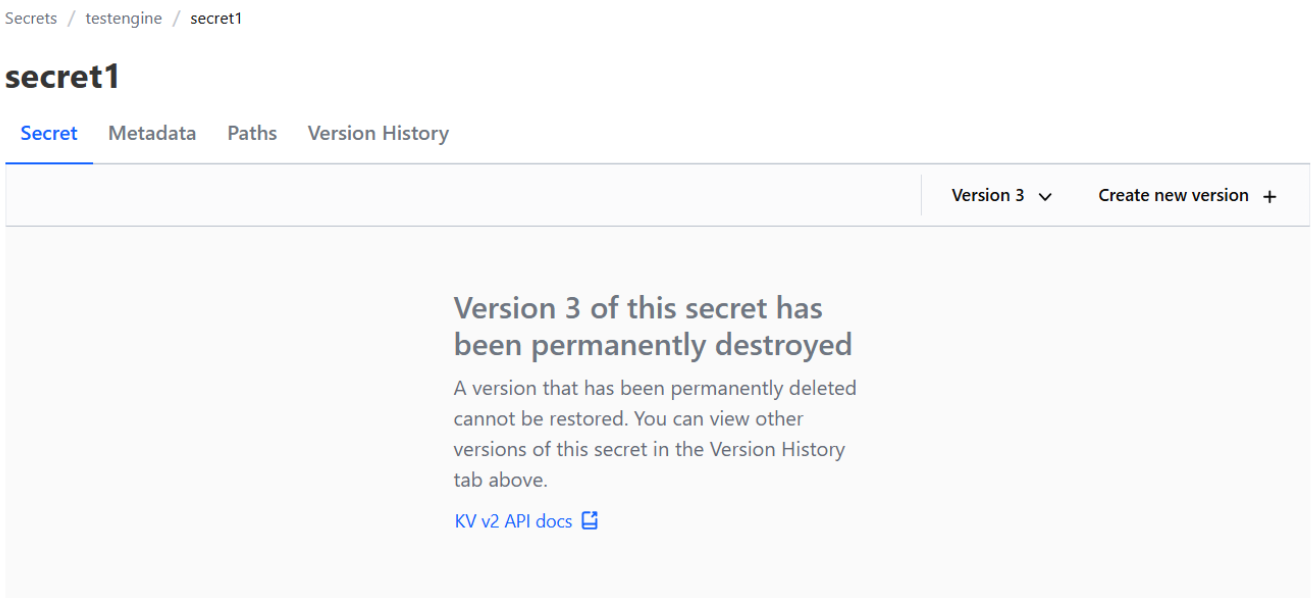
This way secrets are still present but are not readable by any application including any malware or thirdparty attack on the vault.

Destorying

Destroying is the permanent deletion of secret

```
vault kv destroy -mount="testengine" -versions=3  "secret1"
```


© 2026 v1 Sachin Kansal . All rights reserved.



© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/home/master# vault kv get -mount="testengine" -version=3  "secret1"
===== Secret Path =====
testengine/data/secret1

======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T20:36:30.001999761Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          false
version            3

====== Data ======
Key         Value
---         -----
password    dfyabhsfh
username    sachin
root@master-1:/home/master# vault kv destroy -mount="testengine" -versions=3  "secret1"
Success! Data written to: testengine/destroy/secret1
root@master-1:/home/master# vault kv get -mount="testengine" -version=3  "secret1"
===== Secret Path =====
testengine/data/secret1

======= Metadata =======
Key                Value
---                -----
created_time       2026-06-05T20:36:30.001999761Z
custom_metadata    <nil>
deletion_time      n/a
destroyed          true
version            3

root@master-1:/home/master#
```

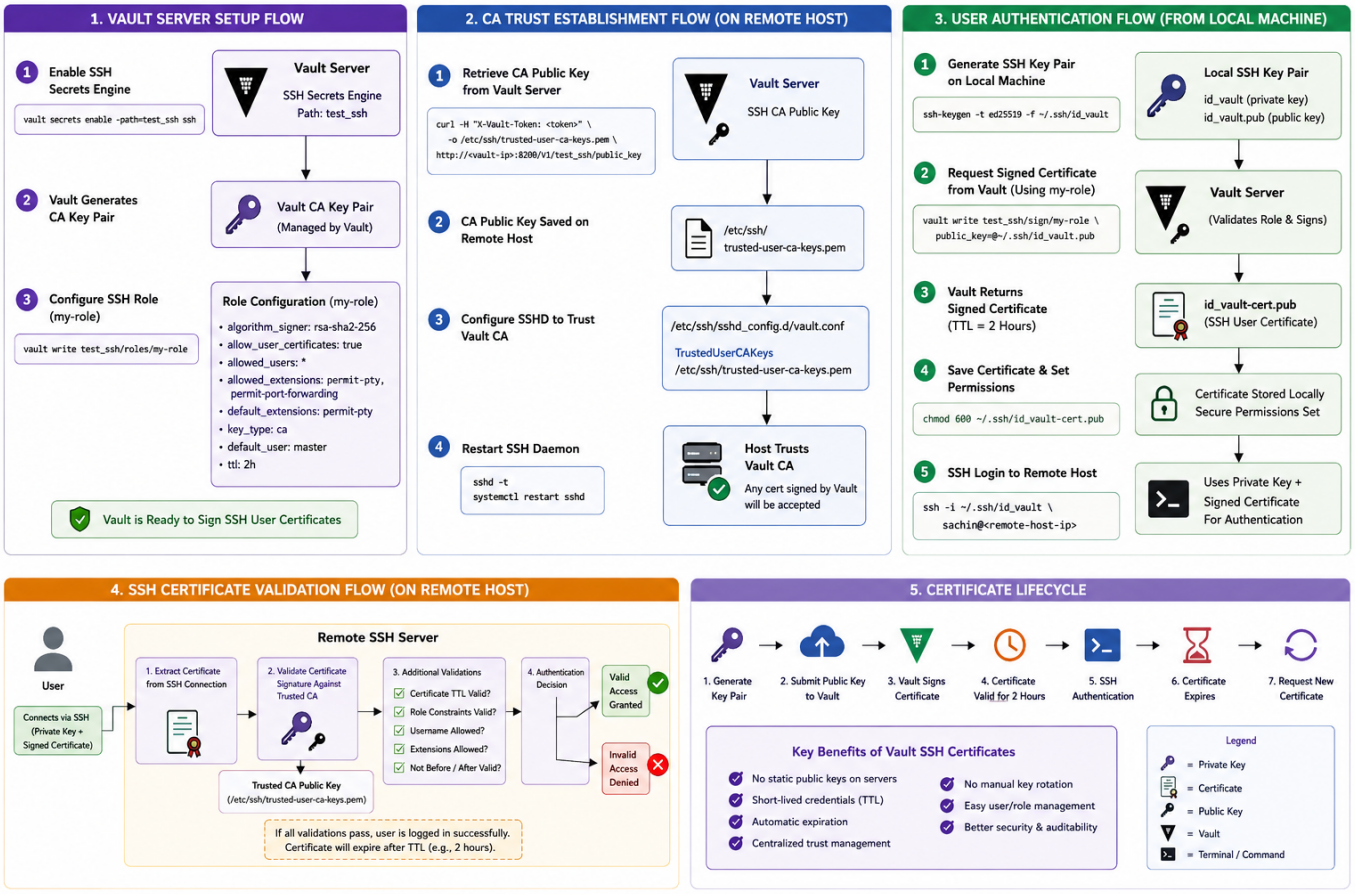
Reading

> to read specific version
>
> ```
> vault kv get -mount="testengine" -version=2  "secret1"
> ```
>
> to read latest current version
>
> ```
> vault kv get -mount="testengine"  "secret1"
> ```
>
> ***Remember the Current version is always latest version***

© 2026 v1 Sachin Kansal . All rights reserved.

Hashicorp VAULT SSH ENGINE

> what we are building



SSH ENGINE: Vault Configuration and Usage

Prerequisites (On Target Machine)

> create the user on target machine

```
useradd -m -s /bin/bash sachin
```

Vault Server Configuration

> Enable SSH Secrets Engine

```
vault secrets enable -path=test_ssh ssh
```

> Configure SSH Role (`my-role`)

```
vault write test_ssh/roles/my-role -<<EOH
{
  "algorithm_signer": "rsa-sha2-256",
  "allow_user_certificates": true,
  "allowed_users": "*",
  "allowed_extensions": "permit-pty,permit-port-forwarding",
  "default_extensions": {
    "permit-pty": ""
  },
  "key_type": "ca",
  "default_user": "master",
  "ttl": "2h"
```


© 2026 v1 Sachin Kansal . All rights reserved.

```
}
EOH
```

Configure Remote Machines

> Retrieve and Trust the CA Public Key

On all remote machines:

```
curl -H "X-Vault-Token: <service_token>" \
     -o /etc/ssh/trusted-user-ca-keys.pem \
     http://192.168.29.10:8200/v1/test_ssh/public_key
```

> Update `sshd` Configuration

Add the trusted CA key to the SSH daemon configuration.

```
/etc/ssh/sshd_config.d# cat vault.conf
TrustedUserCAKeys /etc/ssh/trusted-user-ca-keys.pem
```

> Restart SSH Daemon

```
sshd -t
systemctl restart sshd
```

Local Machine Usage

GENERATE LOCAL SSH KEY PAIR

ssh-keygen -t ed25519 -f ~/.ssh/id_vault

GENERATE SHORT-LIVED SIGNED CERTIFICATE (USING `my-role`)

```
vault write test_ssh/sign/my-role \
    public_key=@~/.ssh/id_vault.pub
```

The output will include the signed certificate.

SAVE THE SIGNED CERTIFICATE

Example command to save the certificate (note: the provided `echo` command saves a specific example cert, the actual command should capture the output of the `vault write` above):

CAPTURING THE ACTUAL OUTPUT IS BETTER:

> USe vault write recommended
>
> ```
> vault write test_ssh/sign/my-role public_key=@~/.ssh/id_vault.pub
> ```
>
> Or echo command:
>
> ```
> echo "ssh-ed25519-cert-v01@openssh.com AAAAI
> ...................
> ..............
> Tw==" \
> > ~/.ssh/id_vault-cert.pub
> ```

Generate Short-Lived Signed Certificate (Using `ca_test` role, if needed)

repeat the vault write command for a role named `ca_test`:

```
vault write test_ssh/sign/ca_test public_key=@~/.ssh/id_vault.pub > ~/.ssh/id_vault-cert.pub
```

SET PERMISSIONS

```
chmod 600 ~/.ssh/id_vault-cert.pub
```


© 2026 v1 Sachin Kansal . All rights reserved.

SSH Access

Use the private key and the signed certificate to authenticate.

```
ssh -i ~/.ssh/id_vault -i ~/.ssh/id_vault-cert.pub sachin@192.168.29.10
```

Note: OpenSSH usually detects the corresponding `-cert.pub` file automatically if the private key is specified, so the following might also work:

```
ssh -i ~/.ssh/id_vault sachin@192.168.29.10
```


© 2026 v1 Sachin Kansal . All rights reserved.

Dynamic Database Secrets

The database is where an application's most valuable asset lives: its data. Breaches are reported almost daily, and leaked personal data leads directly to identity theft and fraud. Every extra hurdle between an attacker and the database matters.

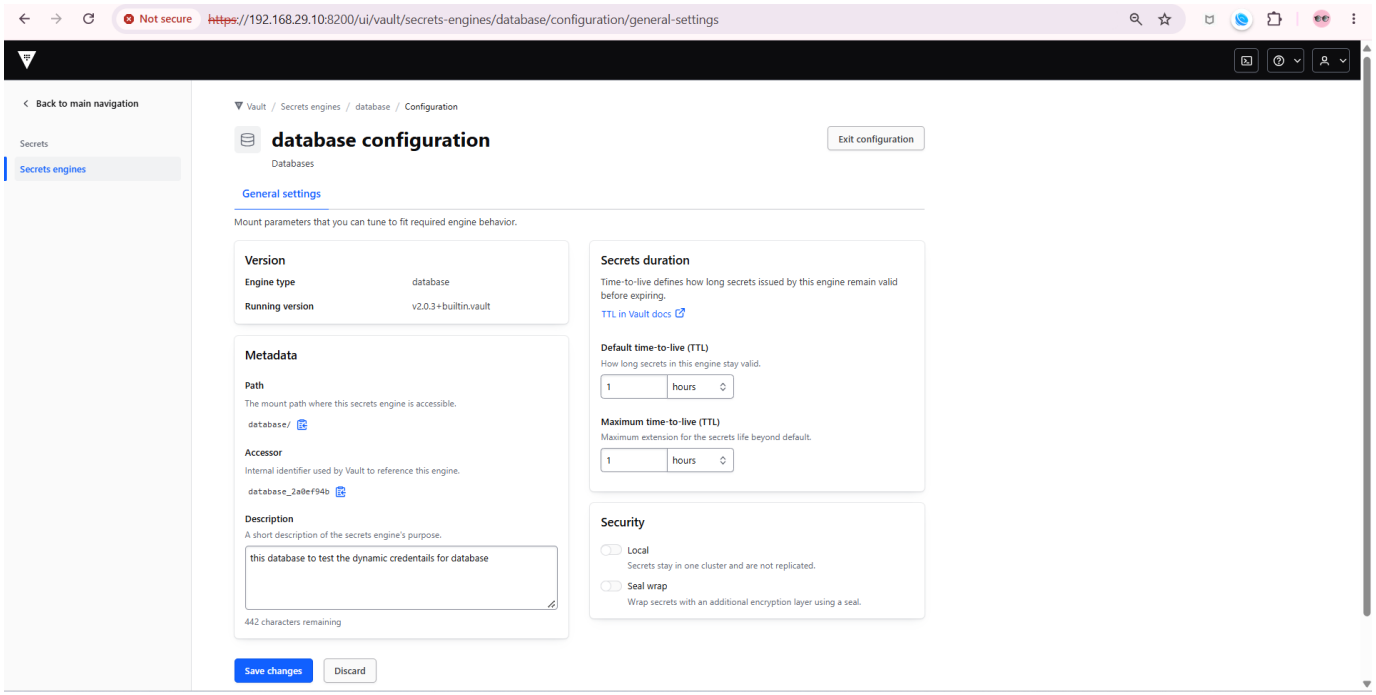
HashiCorp Vault's database secrets engine creates credentials on demand. Each application request gets a unique database user with a limited lifetime (TTL). When the lease expires, Vault automatically drops the user, and the application must request new credentials.

This is more secure than static passwords stored in a Vault KV engine, a Kubernetes Secret, or a shared environment file:

- **No shared or long-lived passwords.** Every credential is unique and temporary, so a leaked one stops working soon after.
- **No human ever needs to see or store the password.**
- **Least privilege by role.** A service that only reads data gets a read-only role and cannot modify data or run DDL commands that change the schema.
- **Audit and revocation.** Every credential request is logged, and any lease can be revoked instantly.

The following steps show how to make your database credentials dynamic.

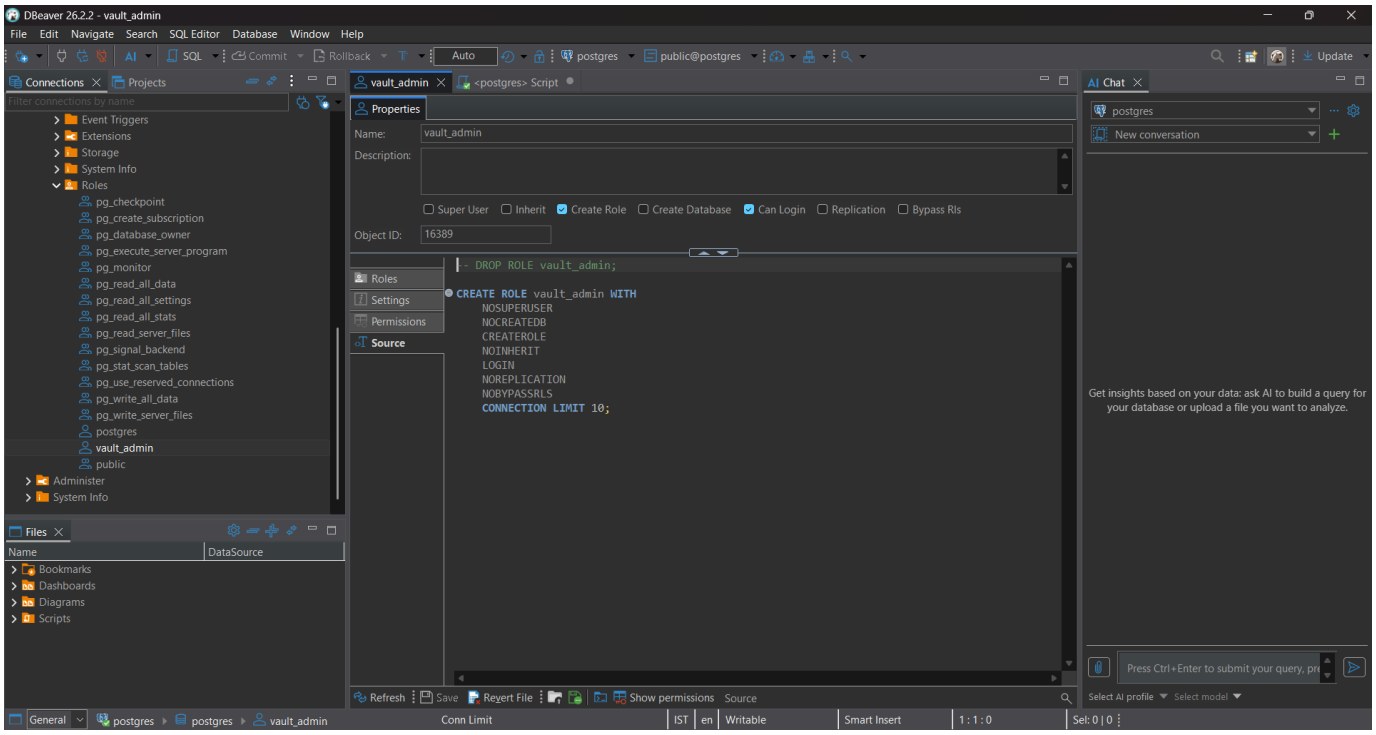
Enable and configure the Engine



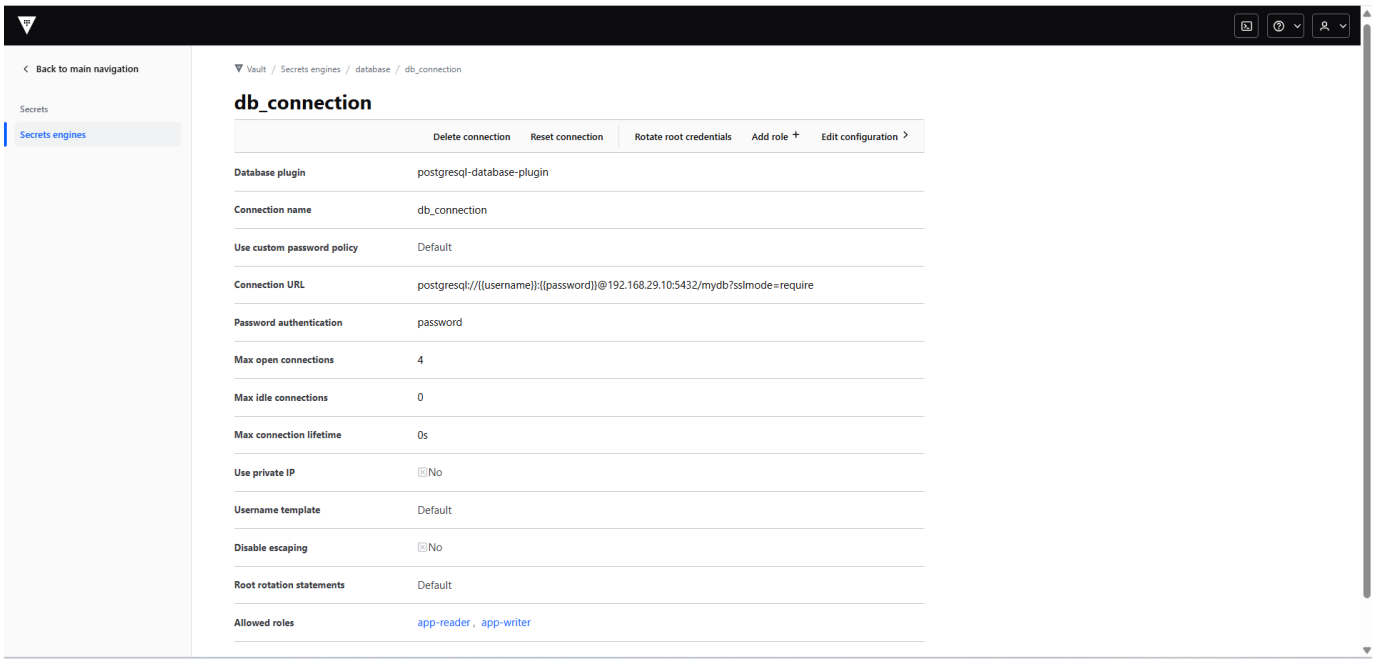
Create the Vault_admin user

> A vault admin user is required in Database side. this user is used to create users and grant them required access as instructed by vault. And don't worry even the credentials for this users are not known by anyone as vault rotates them post db connection is created.

© 2026 v1 Sachin Kansal . All rights reserved.



Create DB connection in Vault



Create Role for app

Creation statement

> SQl statement to create role and grant Select permission to public scheme for all tables.

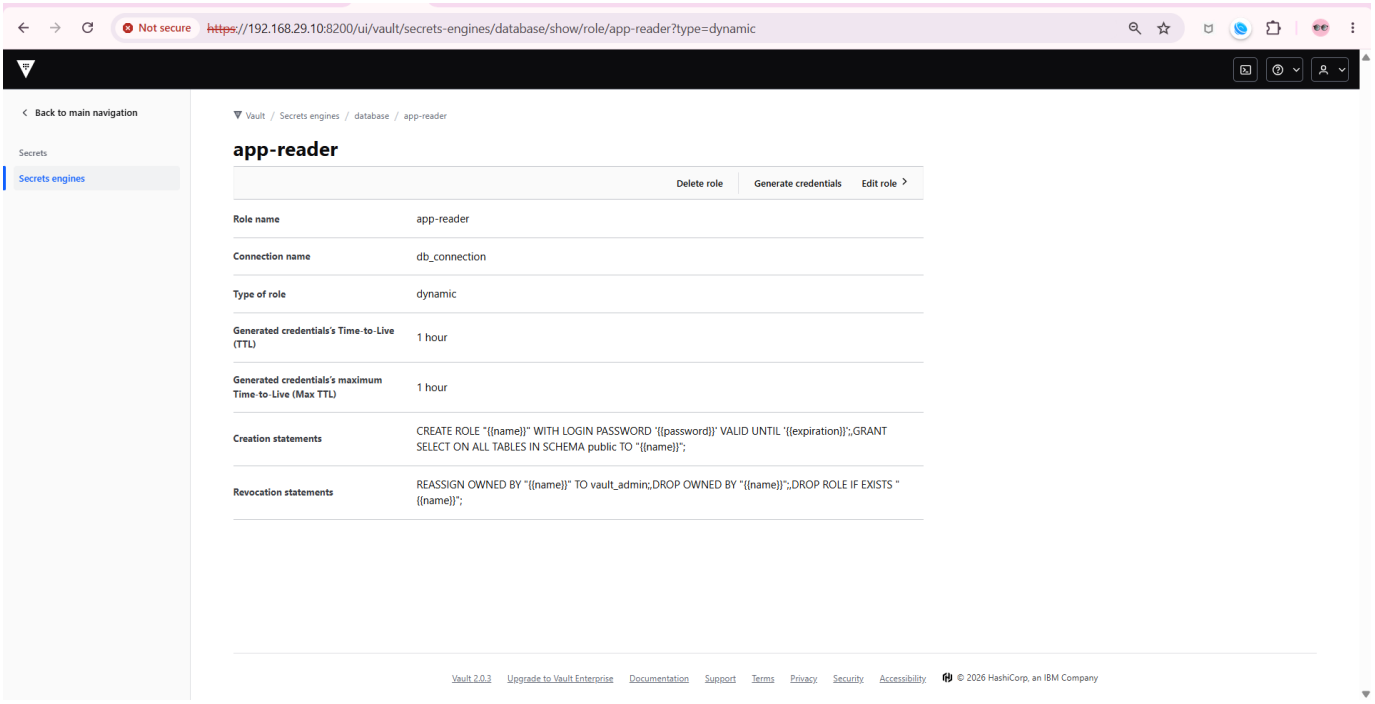
```
CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL '{{expiration}}';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO "{{name}}";
```

Revoke Statement

© 2026 v1 Sachin Kansal . All rights reserved.

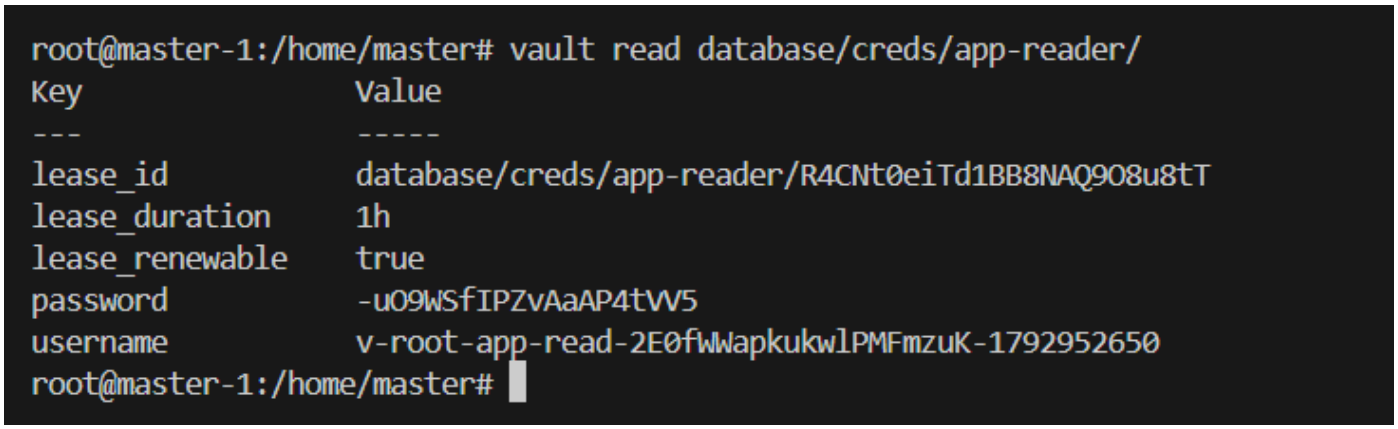
> Sql Statement to revoke the user and grant any object owned by temporary user to assign back to vault_admin.

```
REASSIGN OWNED BY "{{name}}" TO vault_admin;
DROP OWNED BY "{{name}}";
DROP ROLE IF EXISTS "{{name}}";
```



Generate Credentials

```
vault read database/creds/app-reader/
```



> the generated password is shortlived for only 1 hour post that application needs to re-request the new credentials from the vault.

Use Credentials and test

> As it can be seen the user can read data as only select is granted to the role. the Insert command fails

© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/home/master# psql -h 192.168.29.10 -p5432 -d mydb -U v-root-app-read-2E0fWWapkukwlP
K-1792952650
Password for user v-root-app-read-2E0fWWapkukwlPMFmzuK-1792952650:
psql (16.15 (Ubuntu 16.15-0ubuntu0.24.04.1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

mydb=>
mydb=>




mydb=>
mydb=> select * from users;
  user   | userid | age |  sex
--------+--------+-----+--------
 sachin |      1 |  25 | Male
 rahul  |      2 |  30 | Male
 priya  |      3 |  22 | Female
 neha   |      4 |  28 | Female
 amit   |      5 |  35 | Male
 pooja  |      6 |  24 | Female
 rohit  |      7 |  40 | Male
 anjali |      8 |  27 | Female
 vikas  |      9 |  32 | Male
 riya   |     10 |  21 | Female
(10 rows)

mydb=> INSERT INTO users ("user", userid, age, sex) VALUES ('John', '11', '30', 'Male');
ERROR:  permission denied for table users
mydb=>
```


© 2026 v1 Sachin Kansal . All rights reserved.

6. Operations

Migrating Storage Backend

> what we are doing and why ??

Currently we are using filesystem as backened storage that means we are writting everything ti disk which is highly insecure and only okay in dev setups.

So we are going to transfer the data from filesystem to mysql server. where only vault user has acces to backend and noone else.

Cons-> it create a dependency on mysql server meaning server must be healthy for vault to be alive.

> Currently data written in filesystem

```
root@master-1:/mnt/data#
root@master-1:/mnt/data# cd /mnt/data/vault/
root@master-1:/mnt/data/vault# ls
audit  bin  data  deployment.yaml  log  values.yaml
root@master-1:/mnt/data/vault# cd data/
root@master-1:/mnt/data/vault/data# ls
auth  core  logical  sys
root@master-1:/mnt/data/vault/data# cd sys/
root@master-1:/mnt/data/vault/data/sys# ls
counters  expire  policy  token
root@master-1:/mnt/data/vault/data/sys# cd token/
root@master-1:/mnt/data/vault/data/sys/token# ls
_salt  accessor  id  parent
root@master-1:/mnt/data/vault/data/sys/token# cd id/
root@master-1:/mnt/data/vault/data/sys/token/id# ls
_h004d2500979685a0b795e0f009c993ce9e1d4c069f3641eee5d7a4c1bd30555d
_h3f64f064586c369aa9f85f60bcca447f6302707b8574278b407ff36cb0047faa
_h573fbfa00a976c677777918afe3eaaec878985767c67fa88e9e70e929122550e
_h6c551759ffc63278df7562811b246ddc69bde2945700aa70e0badac5aaee42db
_h7c67f7cc91c83f393bdfaa2bbc64add681dc2e71418057ebd9a5c75837f4e447
_h931f1e1d5e283d9b90b1ecada1ec12ea0a123dfbe022f7dc981b3adc5461eba2
_ha00560895cceef04ba7c3ca75a94b1a47e553d24a491c03031f09b2f129e24cd
_ha6e717ac9f0b3ca210064342f64c3c5e9904b384b8a60c61909892f0d05e520e
_hd7819eafcdc1ac99279d540090645cd5c55b52972309397ad2f82346a31f06e3
root@master-1:/mnt/data/vault/data/sys/token/id#
```

Mysql Configuration

> we are not gonna expose this mysql server as it is protected so running on localhost is good. As we are doing on rasberry pie and lots of write on sd card can make the sdcard wear out and shorten its lifespan. hence default /var/lib/mysql/ unsuitable for our cause.

© 2026 v1 Sachin Kansal . All rights reserved.

make sure mysql write write in hard disk or ssd instead of sd card

- step1 update `/etc/mysql/mysql.conf.d\ mysqld.cnf` with new data directory

```
datadir = "/mnt/data/mysql"
```

- step2 update the appArmour config at `/etc/apparmor.d\usr.sbin.mysqld` to grant permission to our new directory

```
# Allow data dir access
  /mnt/data/mysql/ r,
  /mnt/data/mysql/** rwk,
```

and grant ownership to mysql user

```
chown -R mysql:mysql /mnt/data/mysql/
```

- step 3

re-initialize the mysql server

```
mysqld --initialize
```

- step 4 restart the server

```
systemctl restart mysql
```

> you will get locked out and root will not be able to login without password so use the following to get temporary password.
>
> ```
> root@master-1:/mnt/data# mysql
> ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: NO)
>
> grep "temporary password" /var/log/mysql/error.log
> ```
>
> and use the password to login

```
mysql -u root -p
```

> reset password

```
ALTER USER 'root'@'localhost'
IDENTIFIED BY 'XXXXXXXX';
```

------- Now your mysql is writting into the hardisk/ssd instead of default ----------

create db and user inside mysql where vault data will be stored

```
CREATE DATABASE vault;

create user 'vault'@'localhost'
IDENTIFIED BY 'vault-pass';


CREATE ROLE 'vault_role';

GRANT ALL PRIVILEGES ON vault.* TO 'vault_role';

GRANT 'vault_role' TO 'vault'@'localhost';

SET DEFAULT ROLE `vault_role`@`%`
TO `vault`@`localhost`;
```

stop the vault

```
systemctl stop vault
```


© 2026 v1 Sachin Kansal . All rights reserved.

create migration.hcl file

vault migration.hcl file

```
storage_source "file"{
    path = "/mnt/data/vault/data"
}

storage_destination "mysql"{
  address  = "127.0.0.1:3306"
  username = "vault"
  password = "vault-pass"
  database = "vault"
}
```

```
vault operator migrate -config=migration.hcl
```

```
root@master-1:/mnt/data/vault# vault operator migrate -config=migration.hcl
2026-06-05T02:26:42.930+0530 [WARN]  No TLS specified, credentials will be sent in plaintext. To mute this warning add 'plaintext_connection_allowed' with a true va
lue to your MySQL configuration in your config file.
2026-06-05T02:26:44.751+0530 [INFO]  copied key: path=auth/41bbdab3-3428-d927-b454-49f12b94be75/config
2026-06-05T02:26:44.920+0530 [INFO]  copied key: path=core/cluster/local/name
2026-06-05T02:26:44.920+0530 [INFO]  copied key: path=auth/1d2087bd-2ebc-9f4f-78fb-461fc3ada788/user/sachin
2026-06-05T02:26:44.920+0530 [INFO]  copied key: path=core/hsm/barrier-unseal-keys
2026-06-05T02:26:44.920+0530 [INFO]  copied key: path=core/cluster/local/info
2026-06-05T02:26:44.920+0530 [INFO]  copied key: path=core/index-header-hmac-key
2026-06-05T02:26:44.920+0530 [INFO]  copied key: path=core/auth
2026-06-05T02:26:44.920+0530 [INFO]  copied key: path=core/audit
2026-06-05T02:26:44.920+0530 [INFO]  copied key: path=core/cluster/feature-flags
2026-06-05T02:26:44.921+0530 [INFO]  copied key: path=auth/41bbdab3-3428-d927-b454-49f12b94be75/role/myapp
2026-06-05T02:26:45.109+0530 [INFO]  copied key: path=core/keyring
2026-06-05T02:26:45.442+0530 [INFO]  copied key: path=core/local-mounts
2026-06-05T02:26:47.378+0530 [INFO]  copied key: path=sys/token/id/h6c551759ffc63278df7562811b246ddc69bde2945700aa70e0badac5aaee42db
2026-06-05T02:26:47.378+0530 [INFO]  copied key: path=sys/token/id/ha00560895cceef04ba7c3ca75a94b1a47e553d24a491c03031f09b2f129e24cd
2026-06-05T02:26:47.378+0530 [INFO]  copied key: path=sys/token/id/h7c67f7cc91c83f393bdfaa2bbc64add681dc2e71418057ebd9a5c75837f4e447
2026-06-05T02:26:47.378+0530 [INFO]  copied key: path=sys/token/id/ha6e717ac9f0b3ca210064342f64c3c5e9904b384b8a60c61909892f0d05e520e
2026-06-05T02:26:47.379+0530 [INFO]  copied key: path=sys/token/id/h573fbfa00a976c677777918afe3eaaec878985767c67fa88e9e70e929122550e
2026-06-05T02:26:47.379+0530 [INFO]  copied key: path=sys/token/parent/hd7819eafcdc1ac99279d540090645cd5c55b52972309397ad2f82346a31f06e3/ha00560895cceef04ba7c3ca75a
94b1a47e553d24a491c03031f09b2f129e24cd
2026-06-05T02:26:47.467+0530 [INFO]  copied key: path=sys/token/salt
Success! All of the keys have been migrated.
root@master-1:/mnt/data/vault#
```

update the vault.hcl

```
#storage "file" {
#  path = "/mnt/data/vault/data"
#}

storage "mysql" {
  username = "vault"
  password = "vault-pass"
  database = "vault"
  address  = "127.0.0.1:3306"
}
```

restart vault and verify

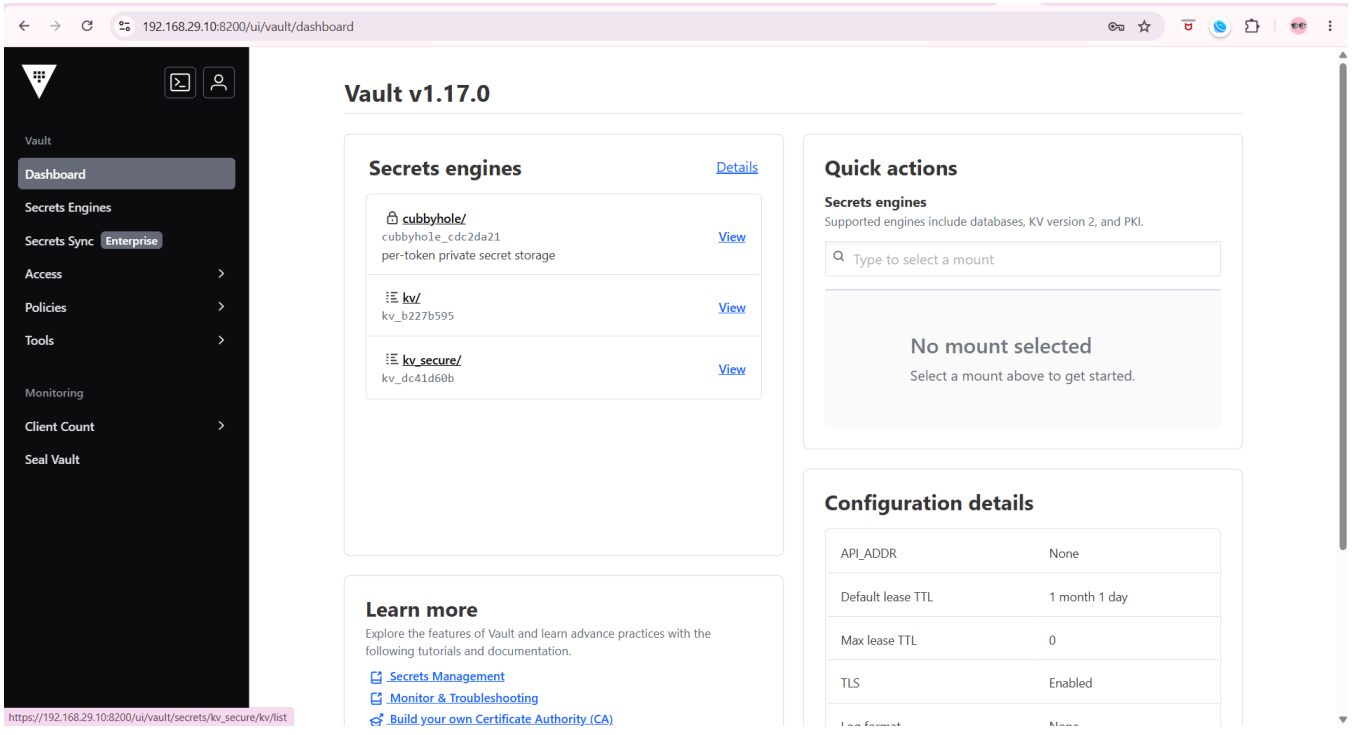
```
systemctl start vault
```


© 2026 v1 Sachin Kansal . All rights reserved.

```
root@master-1:/etc/vault.d# vault status
Key                Value
---                -----
Seal Type          shamir
Initialized        true
Sealed             true
Total Shares       5
Threshold          3
Unseal Progress    0/3
Unseal Nonce       n/a
Version            1.17.0
Build Date         2024-06-10T10:11:34Z
Storage Type       mysql
HA Enabled         false
root@master-1:/etc/vault.d#
```

unseal the vault

```
vault operator unseal
```



© 2026 v1 Sachin Kansal . All rights reserved.

inside the mysql server

> vault table

```
mysql> describe table vault.vault;
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------+
|  1 | SIMPLE      | vault | NULL       | ALL  | NULL          | NULL | NULL    | NULL |   80 |   100.00 | NULL  |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------+
1 row in set, 1 warning (0.01 sec)

mysql>
```


© 2026 v1 Sachin Kansal . All rights reserved.